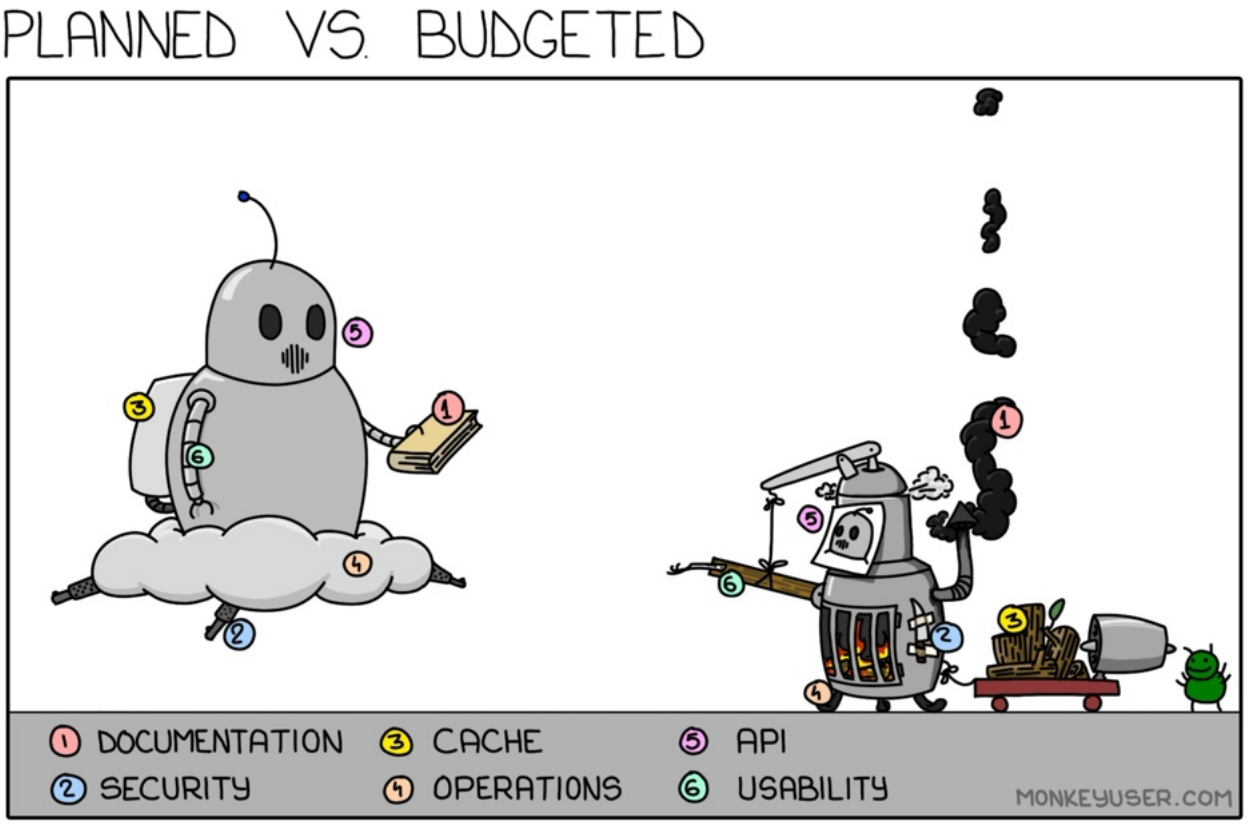
Web Technologies



Web programming (III)

from MVC to Web application engineering

Prof. Sabin Corneliu Buraga – profs.info.uaic.ro/sabin.buraga/
Assoc. Prof. Andrei Panu – profs.info.uaic.ro/andrei.panu/

“Simplicity is a complexity resolved.”

Constantin Brâncuși

(re)discover...

 important aspects regarding Web applications

 in what way can we develop a Web application

 details regarding Web applications architecture

 ”recipes” regarding Web applications development

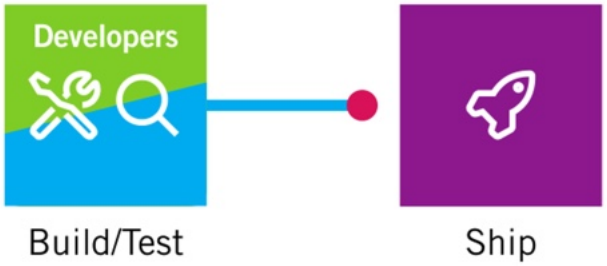
 ways of implementing a Web application

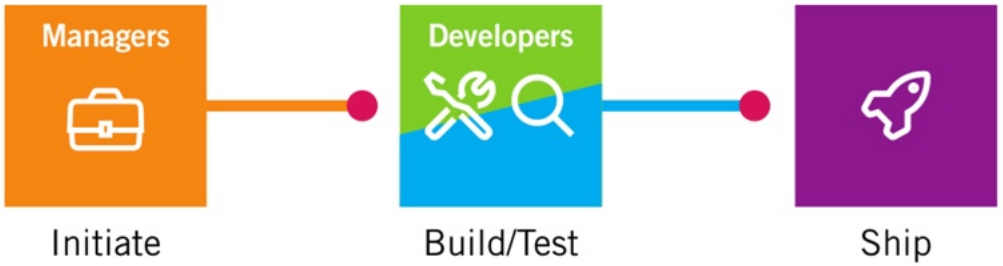
 Web architectures examples

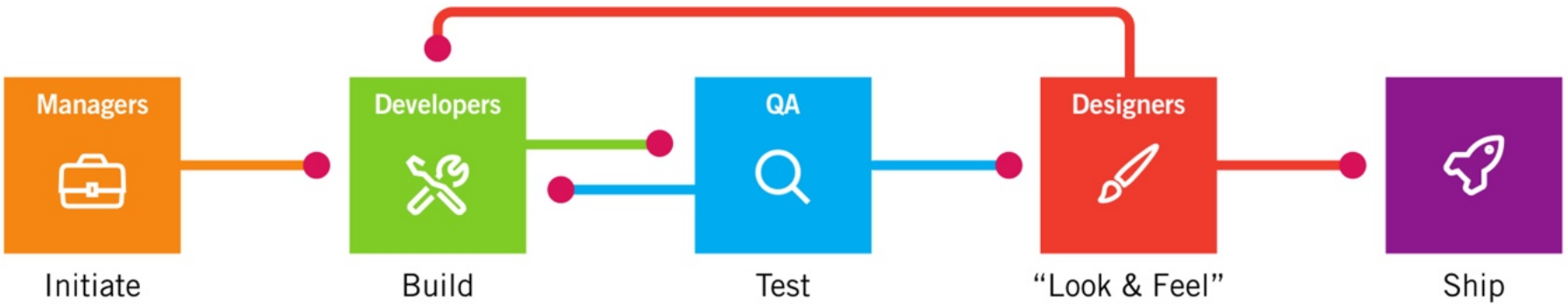
What important aspects do Web applications present?

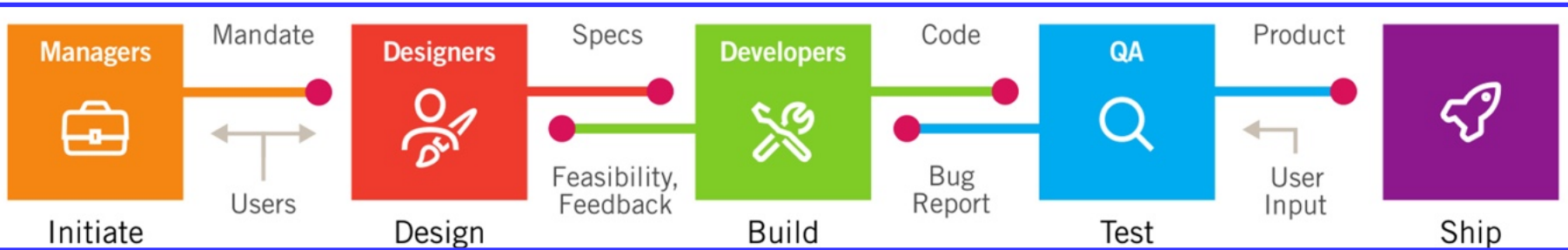
evolution of digital (software) products development

Alan Cooper *et al.*, 2014









Web application quality assurance

correctness and reliability
extending + reusing (modularity)
compatibility
efficiency (performance)
portability

Web application quality assurance

usability
functionality
timeliness
maintainability
security

Web application quality assurance

other aspects of interest:
integrity
reparability
verifiability – including monitoring (logging)
economy

Web application quality assurance

in essence, we must consider:

request retrieval and routing – dispatch

providing basic functionalities – core services

associating software constructs/abstractions (e.g., objects) to the data models – mapping

data management – data

system monitoring & auditing – metrics

adaptation after Matt Ranney, "*What I Wish I Had Known Before Scaling Uber to 1000 Services*", GOTO Chicago 2016

highscalability.com/lessons-learned-from-scaling-uber-to-2000-engineers-1000-ser/

Necessities

a proper specification of aims + requirements

Web application systematic development (in phases)

correct planning of development stages

full monitoring of the entire development process

Necessities

a proper specification of aims + requirements

Web application systematic development (in phases)

correct planning of development stages

full monitoring of the entire development process

- **Web engineering**

In what way can we develop a Web application?

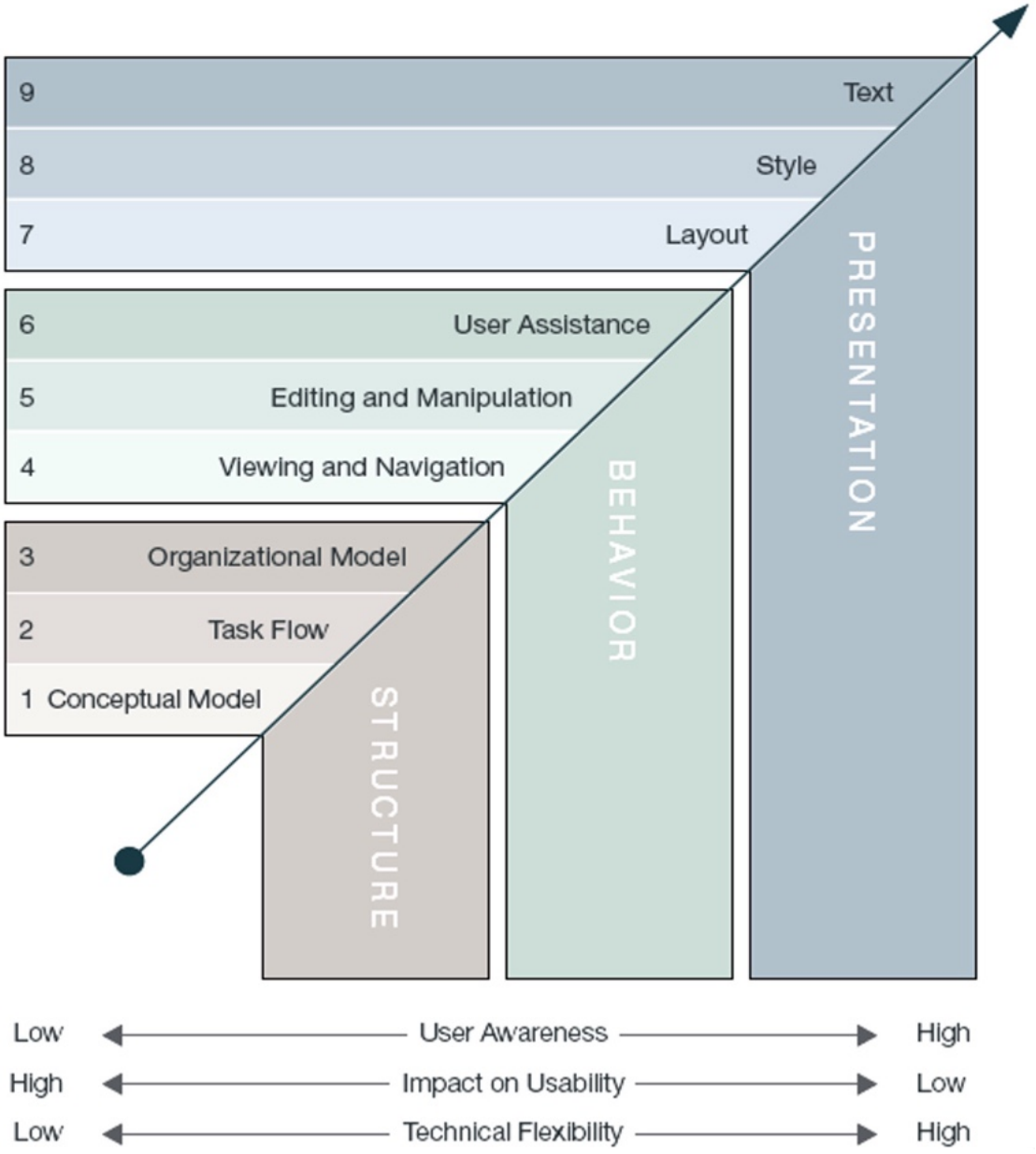
pics.me.me/story-of-every-project-after-3-months-initial-work-plan-57930917.png

modeling

Usually, a methodology is adopted

MDA (Model-Driven Architecture)
is usually preferred

www.omg.org/mda/



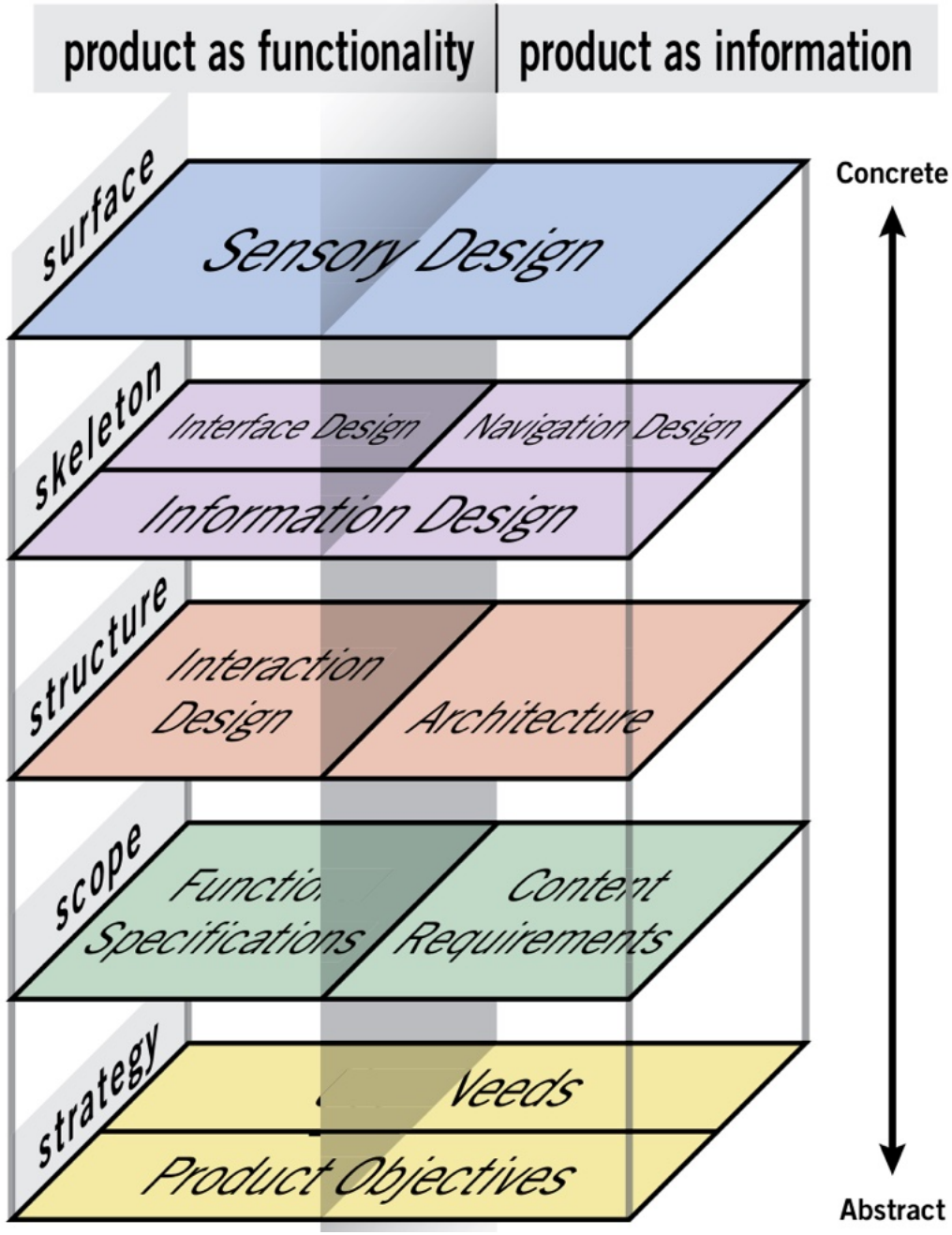
Robert Baxley

web application development

requirements
software design
build (implementation)
testing
deployment
maintenance
evolution

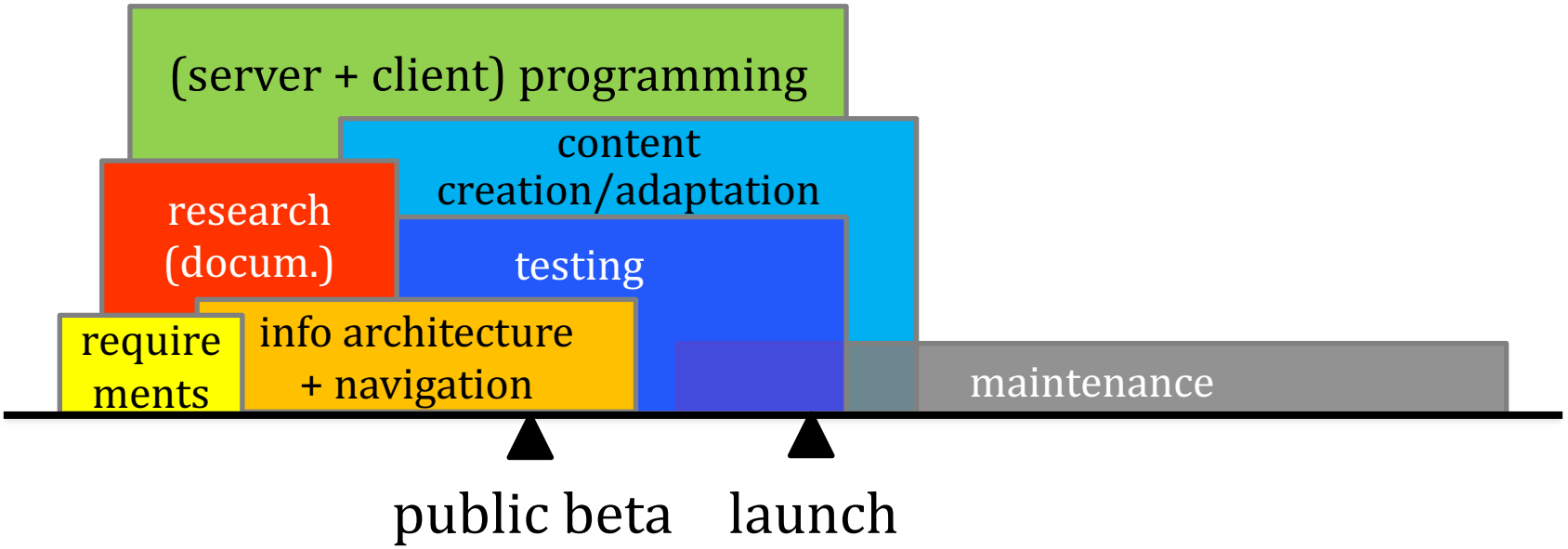
Web application
(software product)

functionality
+
information (content)



Jesse Garrett, *The Elements of User Experience* (2nd Edition), New Riders, 2011

web application development



currently, agile methodologies are preferred

www.infoq.com/process-practices/

www.webpagefx.com/blog/web-design/agile/

web application development: principles

start with needs

do less

design with data

do the hard work to make it simple

iterate. then iterate again

build for inclusion

understand context

build digital services, not Websites

be consistent, not uniform

make things open; it makes things better

an example for **gov.uk** – Paul Downey & David Heath (2013)

requirements

Getting / bidding / negotiating
data (content) and/or source code

copyright
versus
open software (Open Source Licenses)
opensource.org/licenses
+
open data
Creative Commons – **www.creativecommons.org/licenses/**

⚠ aspect of interest: establishing quality standards

requirements

Documenting on the Web application domain

by attracting experts
– subject matter expert (SME) or domain expert –
in the domain of the problem
to be solved by the Web application

requirements: examples

 Vision (big idea)

Basecamp: “project management
and team communication software”

Raindrop.io: “all-in-one bookmark manager”

Wikidata: “a free and open knowledge base
that can be read and edited by both humans and machines”

requirements: examples

Remark: the vision can evolve

Flickr – in the past
"almost certainly the best online photo management and sharing application in the world"
versus
Flickr – in the present
"the best place to be a photographer online"

requirements: examples

Starting points in the Flickr development

initial assumptions:

people like to share memories

using the success of blogging

sharing of (personal) photos, not just notes

providing support for commenting + tagging

new types of requirements

Regarding the content

audience – e.g., internationalization
navigation context
user preferences
permanent availability (7 days, 24 hours/day)
using heterogeneous data sources
support for searching, filtering, recommending
etc.

new types of requirements

User interaction in the Web context

as well as the social Web

content mash-up

“it’s yours to take, re-arrange, and re-use”

new types of requirements

Regarding the execution (runtime) environment

Web browser (in)dependence

user interaction + accessibility
support for various HTML standards: HTML, CSS, SVG,…
multi-device interactivity (responsive Web design)
wired vs. wireless
on-line vs. off-line
security
performance

new types of requirements

Concerning the evolution

users are able to use the Web application
without (re)installing it on a computer/device

new types of requirements: aspects of interest

initially:
providing essential capabilities – less is more

new types of requirements: aspects of interest

initially:
providing essential capabilities – less is more

later versions:
extending the Web application – usually, via a public API (Application Programming Interface),
encouraging the development of user-provided solutions

Some aspects regarding the architecture of Web applications?

architectures

Web applications quality is influenced by the base architecture

Martin Fowler, *Software Architecture Guide* (2019)
martinfowler.com/architecture/

architectures

Web application architecture

📋 **body of code**

that's seen by developers as a single unit

💡 **group of functionality**

that business customers see as a single unit

🎯 **initiative**

that those with the money see as a single budget

architectures

The development of a software architecture considers:

functional requirements

imposed by customers,
visitors,
competition,
decision factors (management),
social/technological growth,
…

architectures

The development of a software architecture considers:

qualitative factors

usability
performance
security
data/code reuse
etc.

architectures

The development of a software architecture considers:

techn(olog)ical aspects

hardware/software platform (e.g., operating systems)
middleware infrastructure
available services – usually, via public APIs
programming language(s)
legacy systems
…

architectures

The development of a software architecture considers:

experience

adopting existing architectures and platforms
design patterns
using specific solutions: libraries, frameworks, tools,…
project management
etc.

client(s)
firewall
proxy
middleware
Web server(s)
Web application server(s)
frameworks, libraries, other components
persistent storage server(s) – e.g., databases
multimedia content server(s)
content management server(s) – e.g., CMS, wiki
legacy applications/systems

typical “ingredients”

client(s)

firewall

proxy

middleware

Web server(s)

Web application server(s)

frameworks, libraries, other components

persistent storage server(s) – e.g., databases

multimedia content server(s)

content management server(s) – e.g., CMS, wiki

legacy applications/systems

eventually, using cloud services – **cloud computing**

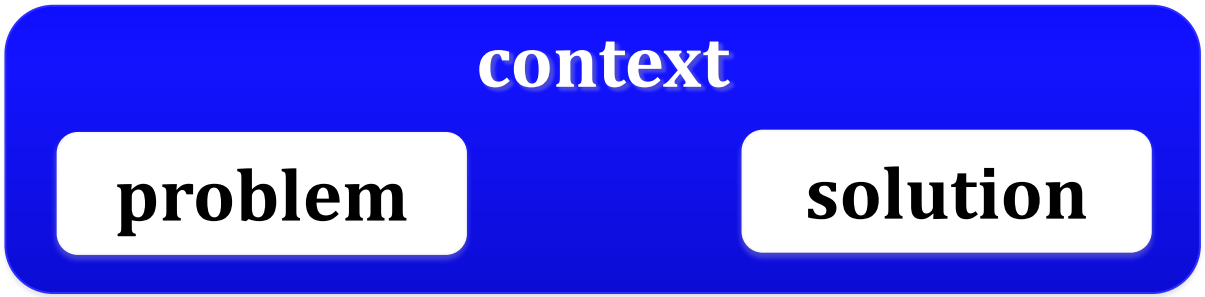
computing resources and data sharing on demand, with other computers/devices, based on Internet technologies (hosting, scalable infrastructure, parallel processing, monitoring, …)

Are there certain “recipes” regarding Web application development?

design

Pattern

rule denoting a relation
between a **context**, a **problem**, and a **solution**
considering the point of view of an expert



design

A pattern specification and/or “identification”
could be found on various tiers:

data presentation – UI, user interaction, visualization,…
processing – business logic, scripting, etc.
component integration – code library development
data storage – database queries, database design,…

design

Examples of pattern collections
(pattern repositories)

concerning software design
wiki.c2.com/?DesignPatterns
patterns of enterprise application architecture
martinfowler.com/eaaCatalog/
user interaction (Adele – a repository of
publicly available design systems and pattern libraries)
adele.uxpin.com

design

Web Patterns

Model View Controller
Page Controller
Front Controller
Template View
Transform View
Application Controller

M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley (2003)

design

Session State Patterns

Client Session State
Server Session State
Database Session State

M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley (2003)

design

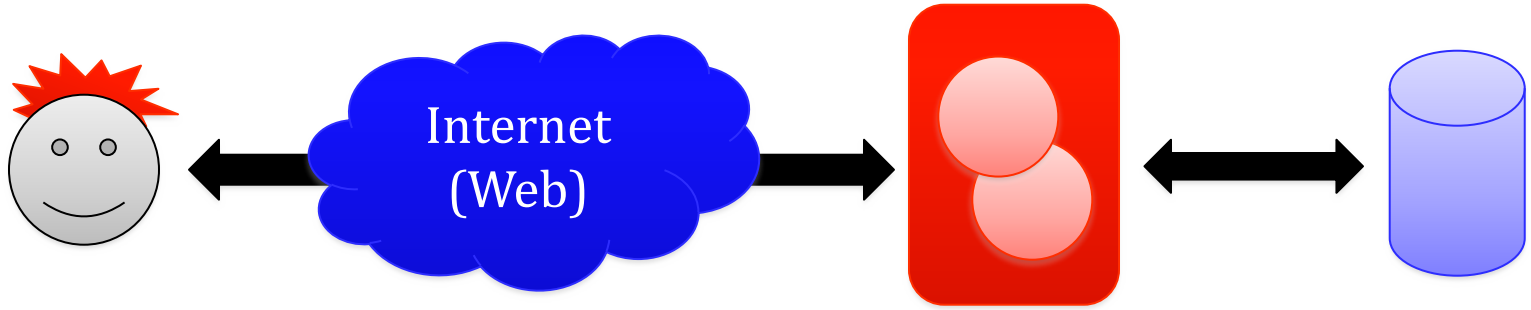
Data Source Architectural Patterns

Table Data Gateway
Row Data Gateway
Active Record
Data Mapper

M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley (2003)

Web application = **interface** + **program** + **content** (data)

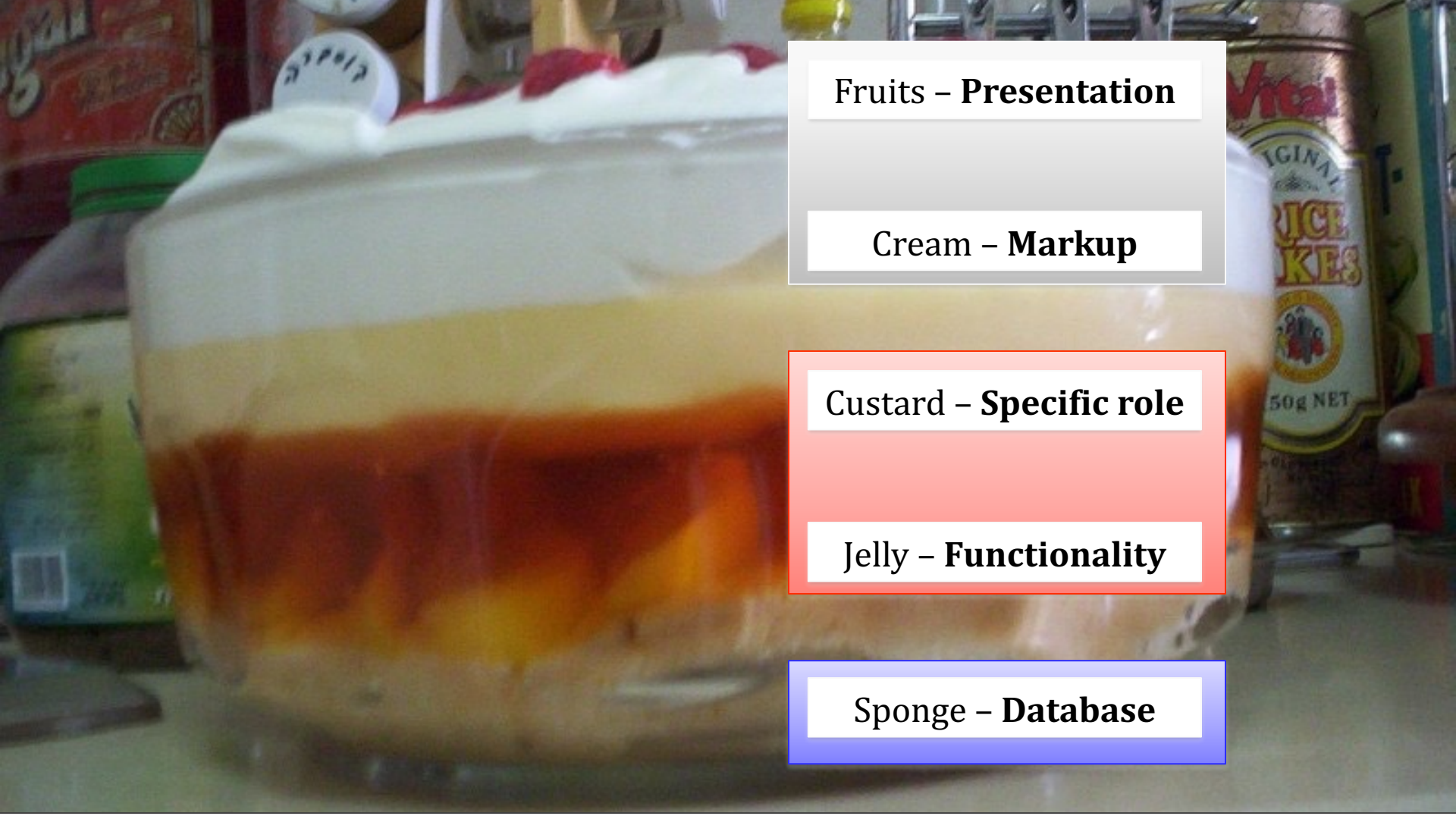
3-tier application



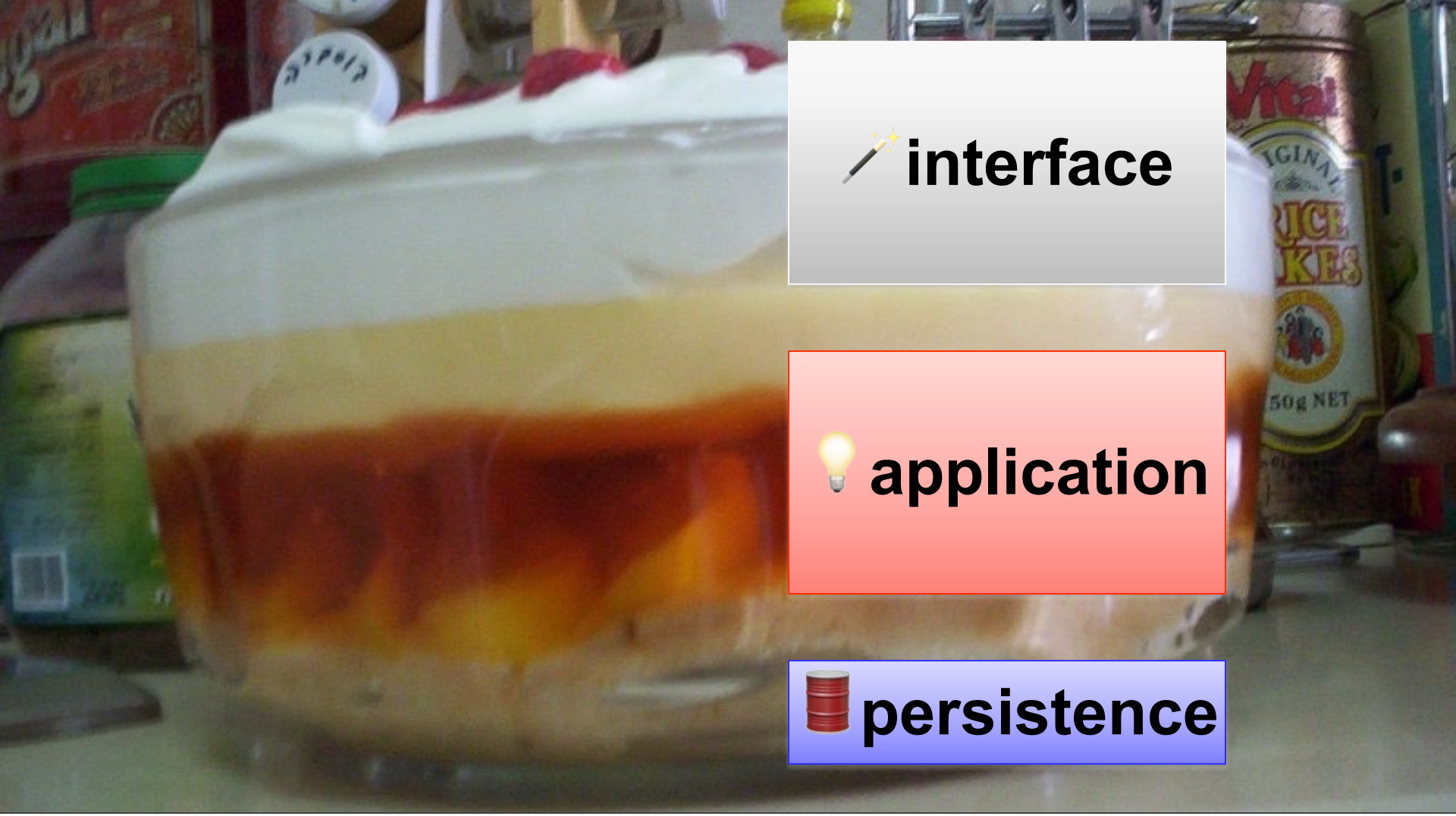
Client
(interface)

Application Server
(application)

Storage
(persistence)



C. Henderson, *Scalable Web Architectures*, Web 2.0 Expo, 2007: iamcal.com/talks/



C. Henderson, *Scalable Web Architectures*, Web 2.0 Expo, 2007: iamcal.com/talks/

web architectures

Data structure model is detached by
the processing model
(business logic, application control)
and the data presentation model (Web interface)

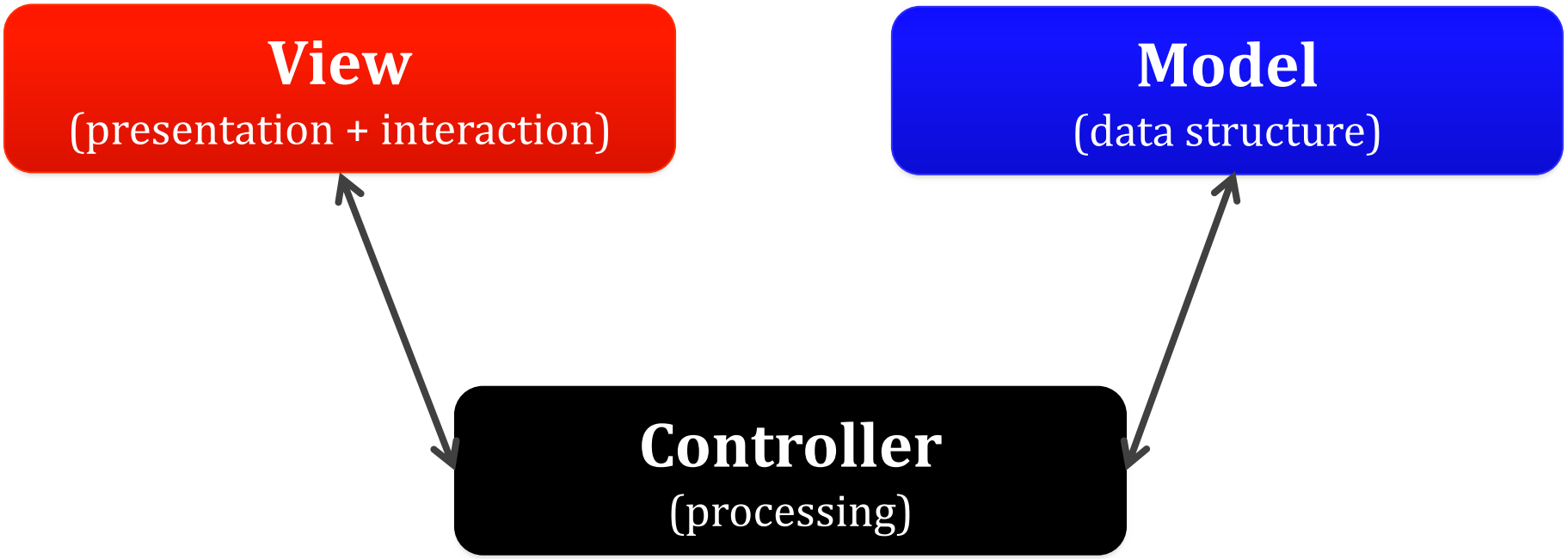
principle: **separation of concerns**

web architectures: mvc

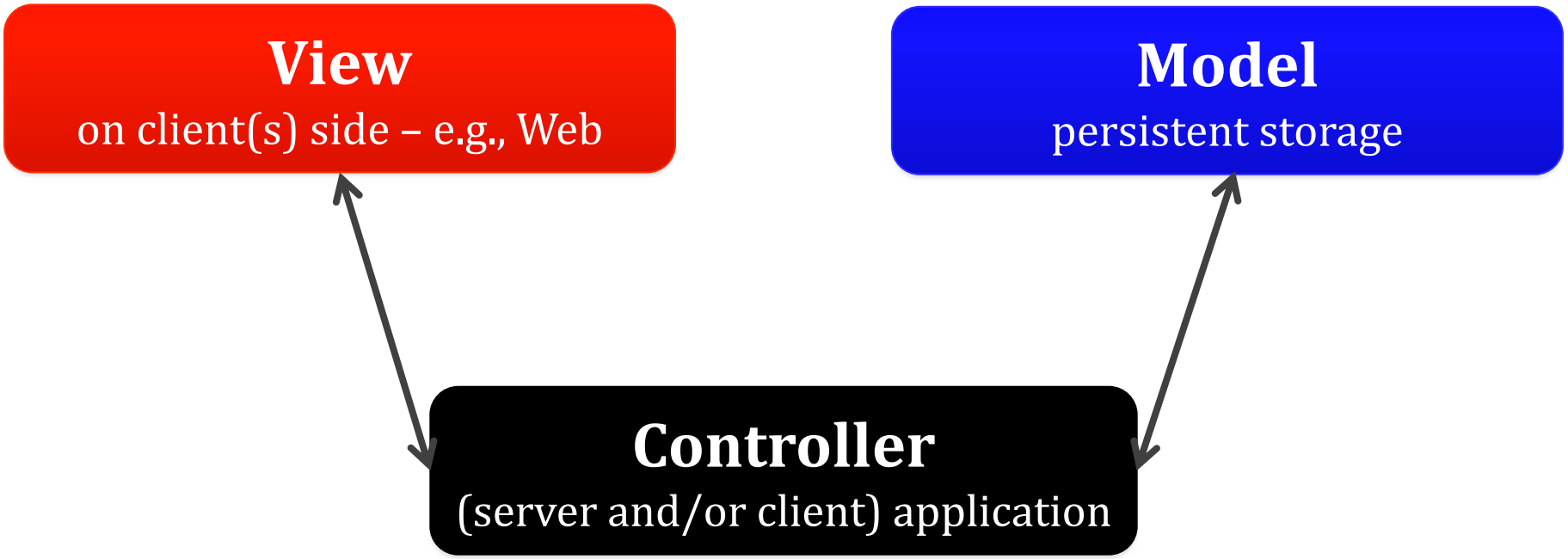
Most Web applications are developed by adopting **MVC** (**Model-View-Controller**)

Trygve Reenskaug, 1979
folk.universitetetioslo.no/trygver/1979/mvc-2/1979-12-MVC.pdf

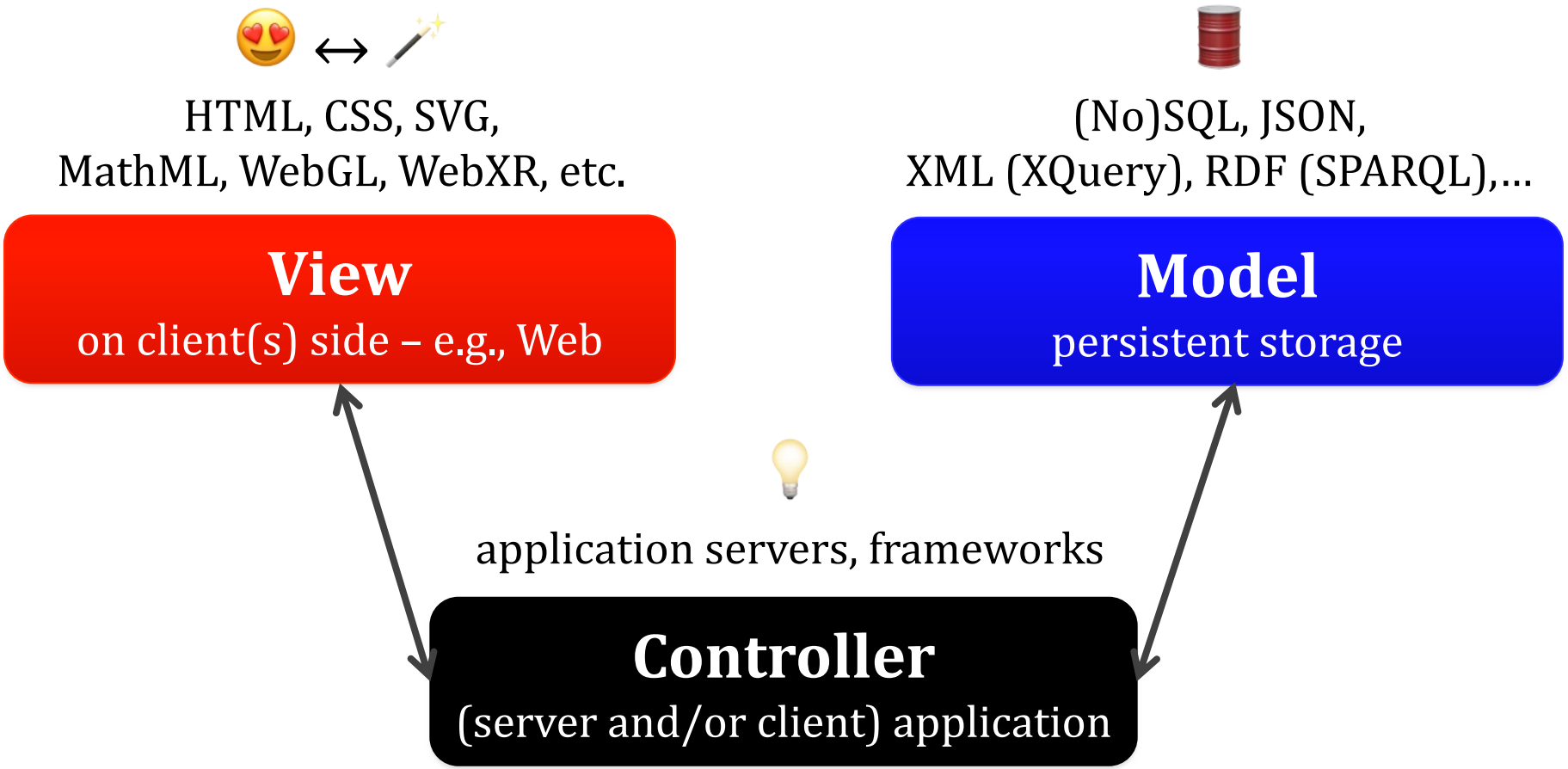
web architectures: mvc



web architectures: mvc



web architectures: mvc



web architectures: mvc

Could be implemented in a non-object-oriented language

encouraged/imposed by specific Web frameworks

various examples:

ASP.NET MVC (C# *et al.*), Catalyst (Perl), Conduit (Dart), Django (Python), Express (Node.js), Grails (Groovy), Laravel (PHP), Lift (Scala), Rails (Ruby), Rocket (Rust), Yesod (Haskell), Spring (Java), Wt (C++)

web architectures: mvc

Controller

responsible with the retrieval of requests from the client (HTTP requests – e.g., GET, POST,… – issued based on user actions)

web architectures: mvc

Controller

responsible with the retrieval of requests from the client (HTTP requests – e.g., GET, POST,… – issued based on user actions)

manages the resources required for request fulfilment

web architectures: mvc

Controller

responsible with the retrieval of requests from the client (HTTP requests – e.g., GET, POST,… – issued based on user actions)

manages the resources required for request fulfilment

usually, will call a model according to the required action and, after that, will select the corresponding view

web architectures: mvc

Model

software-managed resources
– users, messages, products, etc. – have specific models

web architectures: mvc

Model

software-managed resources
– users, messages, products, etc. – have specific models

signifies data + rules (i.e. constraints) concerning data

- **concepts** managed by the Web application

web architectures: mvc

Model

software-managed resources
– users, messages, products, etc. – have specific models

signifies data + rules (*i.e.* constraints) concerning data
- **concepts** managed by the Web application

offers the controller a representation of the desired data
and is responsible with the validation data to be stored

web architectures: mvc

View

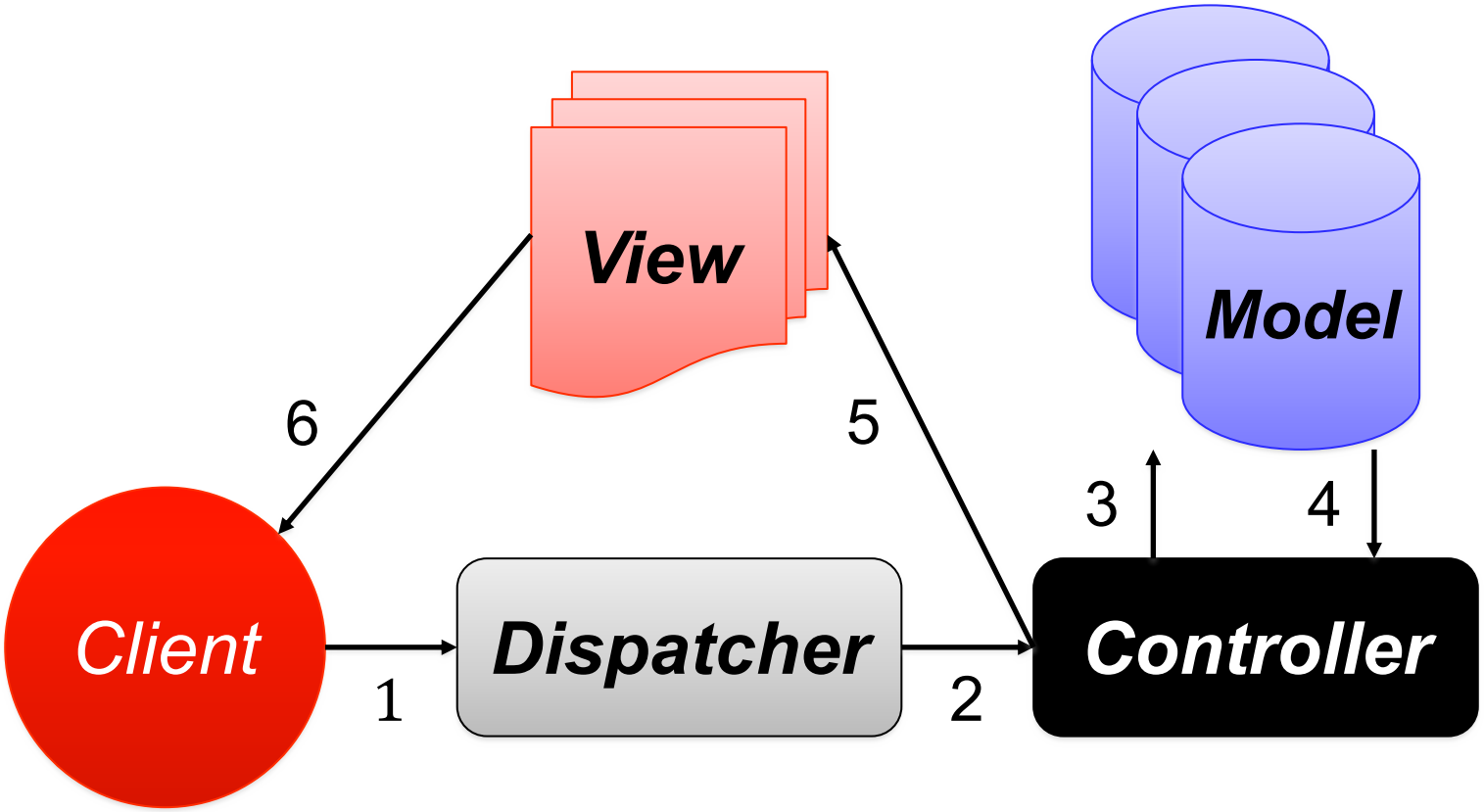
provides various ways of presenting the data offered by the model via controller

web architectures: mvc

View

provides various ways of presenting the data
offered by the model via controller

multiple views can exist,
a specific one being chosen by the controller



typical steps:
(1) request sent by client – e.g., Web browser,
(2) routing request to the *controller*,
(3) choosing a *model*, (4) providing the desired data,
(5) selecting a *view*, (6) content directed to the client

web architectures: mvc

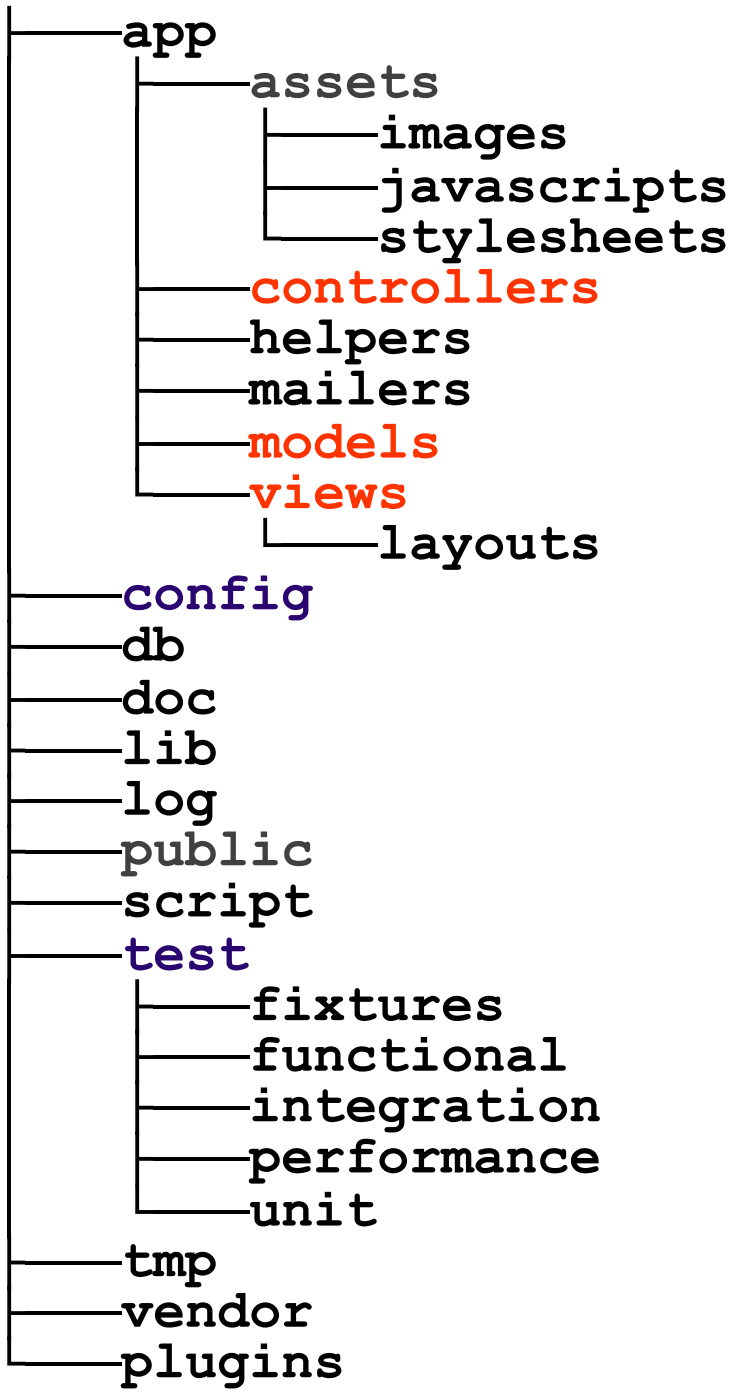
A Web application's generic architecture will consist of a set of resources regarding the controller, model, and view

web architectures: mvc

A Web application's generic architecture will consist of a set of resources regarding the controller, model, and view

usually, the used Web framework will enforce a specific file structure of the application to be implemented

advanced

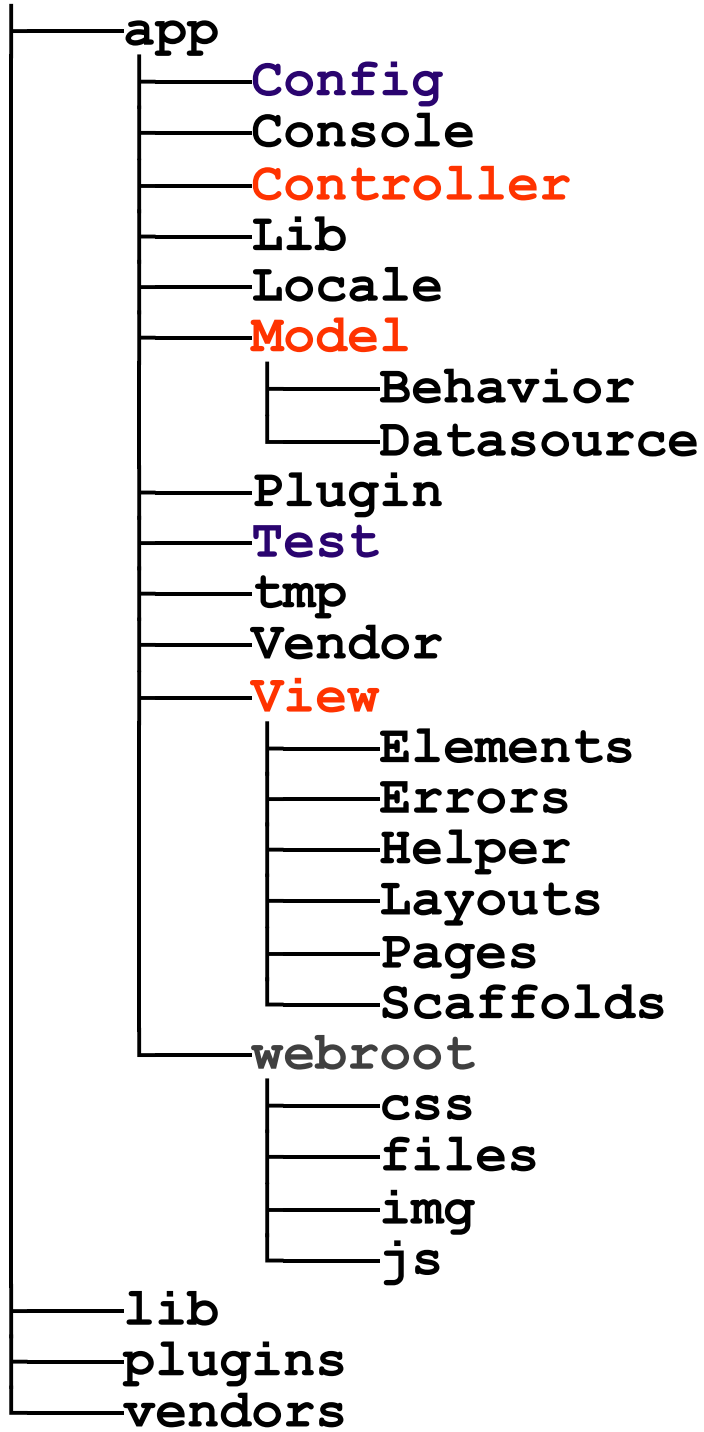


a Web application “skeleton”
created with Ruby on Rails
rubyonrails.org

advanced

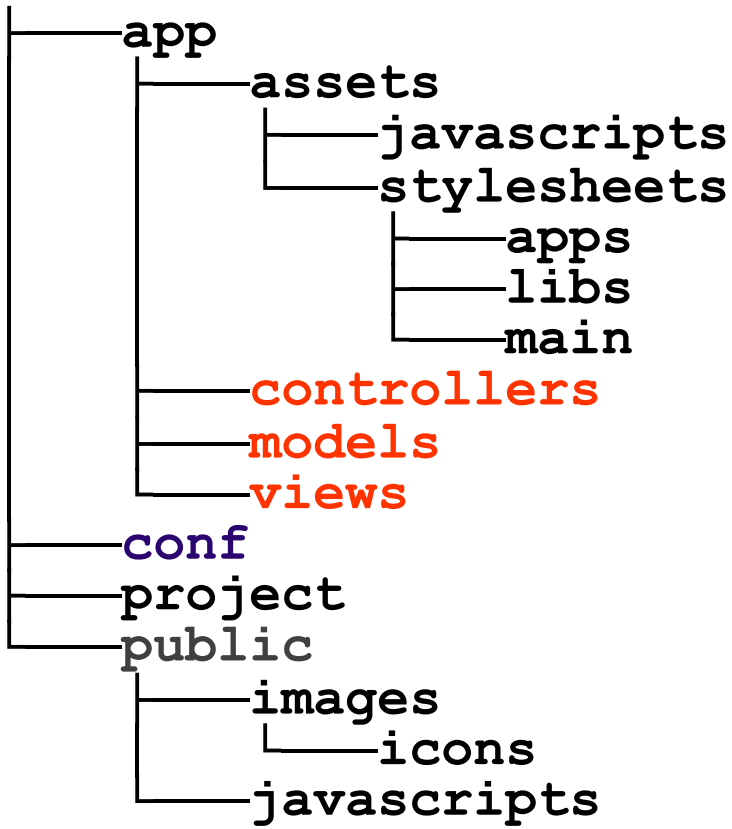
the directory structure of a Web application built with CakePHP framework

cakephp.org



others: CodeIgniter, FuelPHP, Laravel, Symfony, Yii

advanced



the directory structure generated for a Web application using the Play framework for Java and Scala

www.playframework.org

advanced

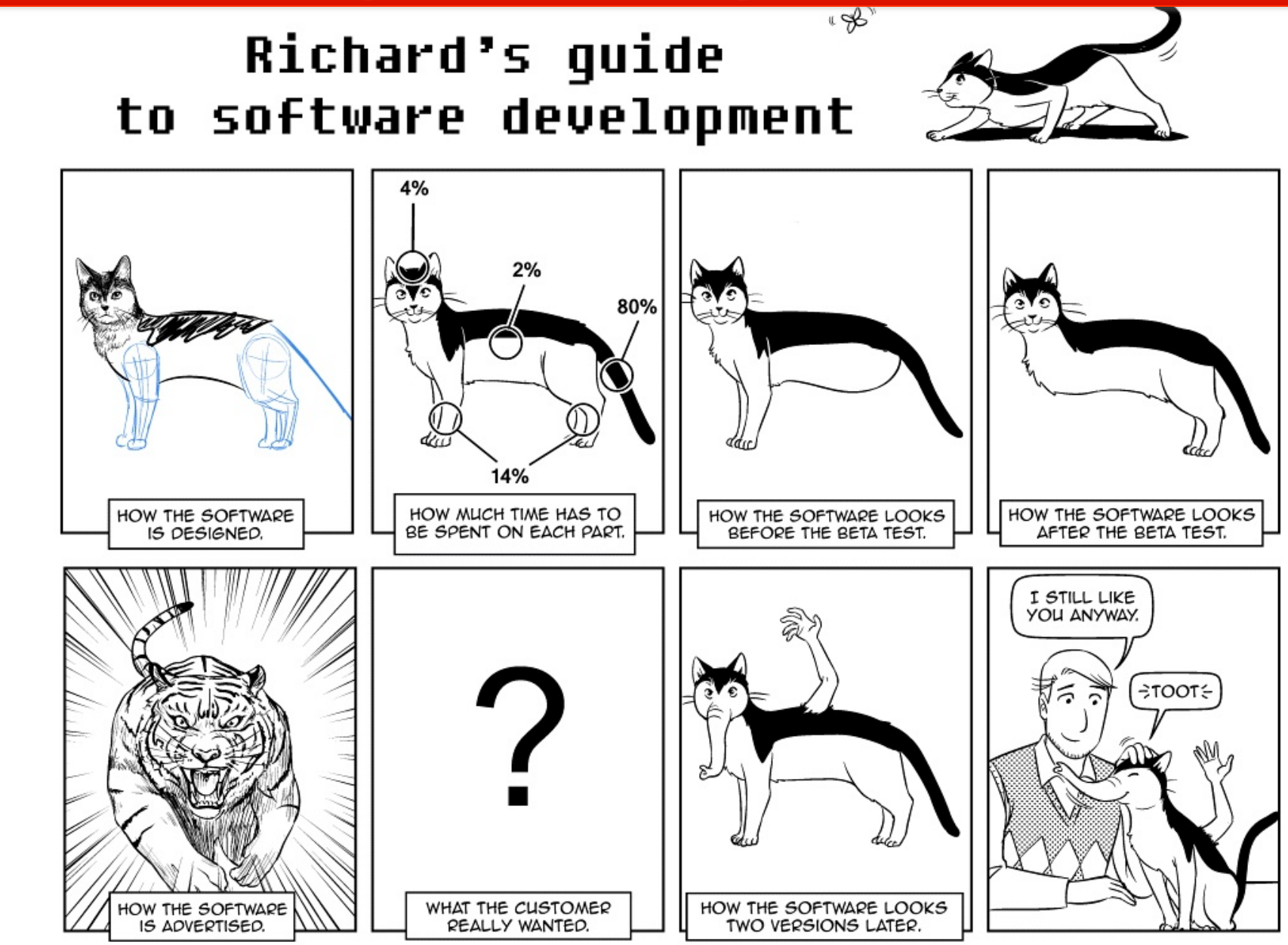
web architectures: mvc

Derived variants:

HMVC (Hierarchical Model-View-Controller)
MVP (Model View Presenter)
MVVM (Model View ViewModel)

herbertograca.com/2017/08/17/mvc-and-its-variants/

(instead of) break



By what means
can a Web application be implemented?

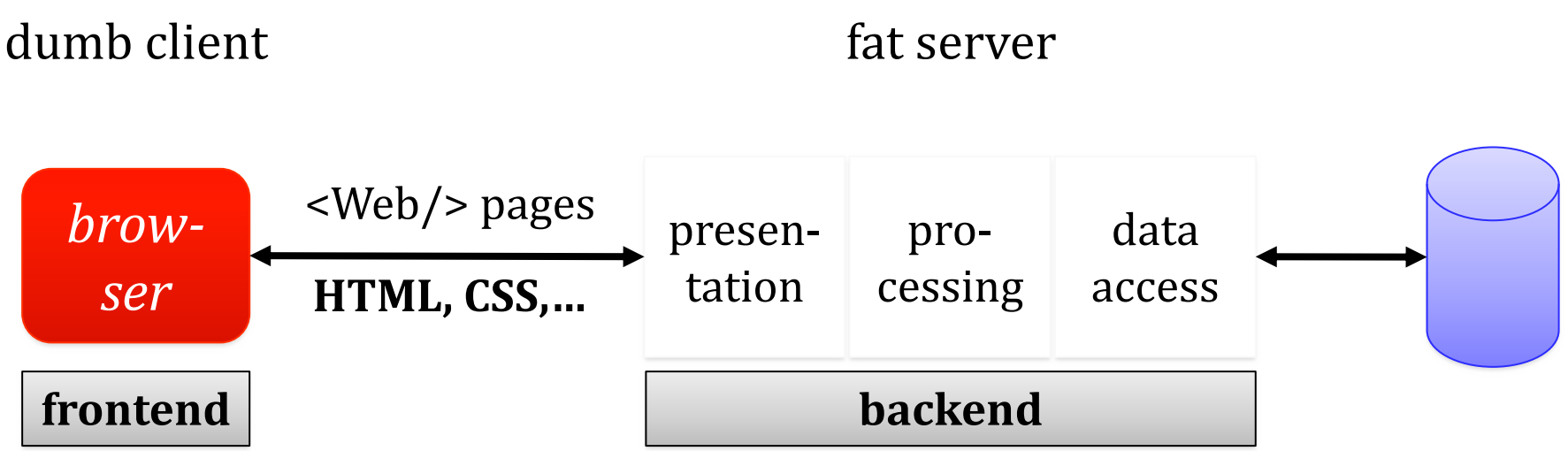
implementation

Web application server

purpose:
optimizing the development processes
of complex Web applications

could encourage or impose an architectural approach for Web application development – e.g., MVC or variants

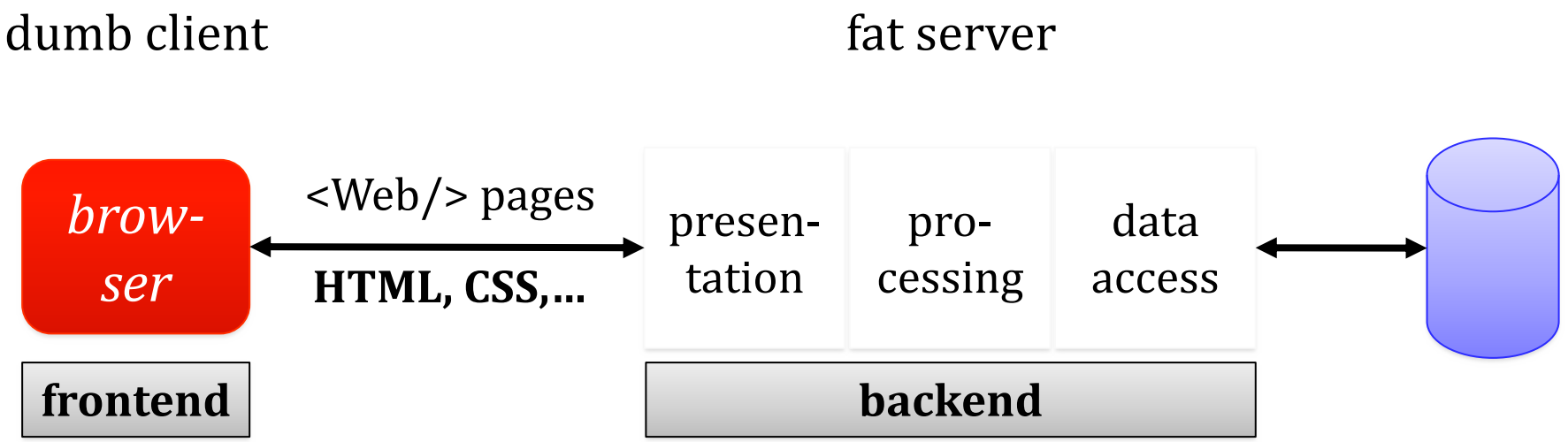
Web application architecture: **classic MV* approach**



content generation at Web server level
usually, **monolithic application**

www.leaseweblabs.com/2013/10/api-first-architecture-fat-vs-thin-server-debate/

Web application architecture: **classic MV* approach**



design principle:

layers of isolation

changes made in a particular layer have no impact on or do not affect the components in another layer

advanced

implementation

Framework

facilitates complex Web application development, by simplifying usual operations (e.g., access to databases, caching, code generation, session management, access control) and/or encouraging the source code reuse

advanced

implementation

Various frameworks facilitating Web application development on the backend:

ASP.NET: ASP.NET Core MVC
Java: Play, Spring, Sling, Struts, Tapestry, Wicket
JavaScript (Node.js): Express, Koa, LoopBack, Meteor
Perl: Catalyst, Dancer, Mojolicious
Python: CherryPy, Flask, Pylons Project, web2py
Ruby: Padrino, Rails, Sinatra
Rust: Actix Web, Loco, Gotham, Rocket

implementation

Web library

a collection of reusable computational resources
– i.e., data structures + code –
providing specific functionalities (behaviors)
implemented in a given programming language

implementation

Web library

a collection of reusable computational resources
– i.e., data structures + code –
providing specific functionalities (behaviors)
implemented in a given programming language

could be referred by other source code (software):
application server, framework, library,
service, API, or Web component

Open source libraries – examples:

Apache PDFBox – **pdfbox.apache.org**
Beautiful Soup – **www.crummy.com/software/BeautifulSoup**
D3.js – **d3js.org**
Expat – **libexpat.github.io**
ImageMagick – **www.imagemagick.org**
libcurl – **curl.haxx.se**
OpenCV – **opencv.org**
RDFlib – **rdflib.readthedocs.org**
SQLAPI++ – **www.sqlapi.com**
TensorFlow.js – **www.tensorflow.org/js/**
zlib – **www.zlib.net**

implementation

Web service

software – remotely used by other applications/services – providing a specific functionality, frequently through an API (Application Programming Interface)

its implementation must not be known
by the programmer that invokes the service

advanced

implementation

SDK (**Software Development Kit**)

encapsulates the API functionalities into a library
(implemented in a programming language,
for a specific software/hardware platform)

API façade pattern

advanced

implementation

SDK (**Software Development Kit**)

example:
Octokit (for .NET, Objective-C, Ruby,…)
provided by Github – **docs.github.com/en/rest/overview/libraries**

advanced

implementation

Web component

part of a Web application
that includes a set of related features

e.g., calendar, news feed reader, URL sharing button

advanced

implementation

Web component

development based on a suite of technologies implemented by the browser – Web Components
developer.mozilla.org/docs/Web/Web_Components
open-wc.org

eventually, a JavaScript library or framework can be used, like hybrids, OMI, Lit, snuggsi, Stencil

implementation

Widget

application – stand-alone or included into a container
(e.g., an HTML document) –
offering a specific functionality

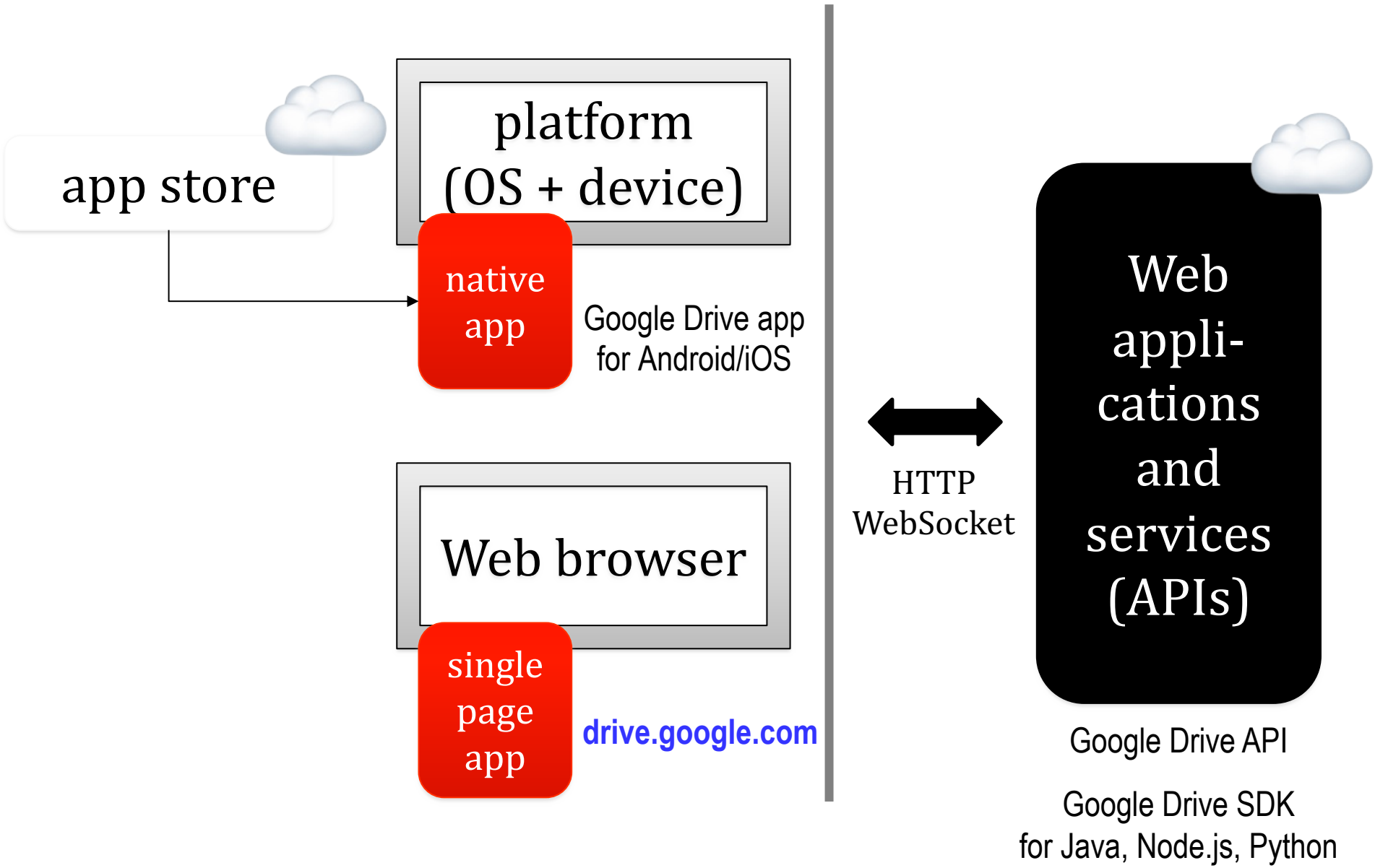
runs on a client (a platform provided by
an operating system and/or a Web browser)

implementation

(Web) app

an installable (Web) application
that uses the APIs provided by a platform:
browser, application server, operating system, device,…

advanced



adapted from Adrian Colyer (2012)

implementation

Add-on

a generic name for browser-associated applications
(extensions, visual themes, dictionaries,
Web search engine interfaces, plug-ins, etc.)

examples: addons.mozilla.org

development

Using development environments

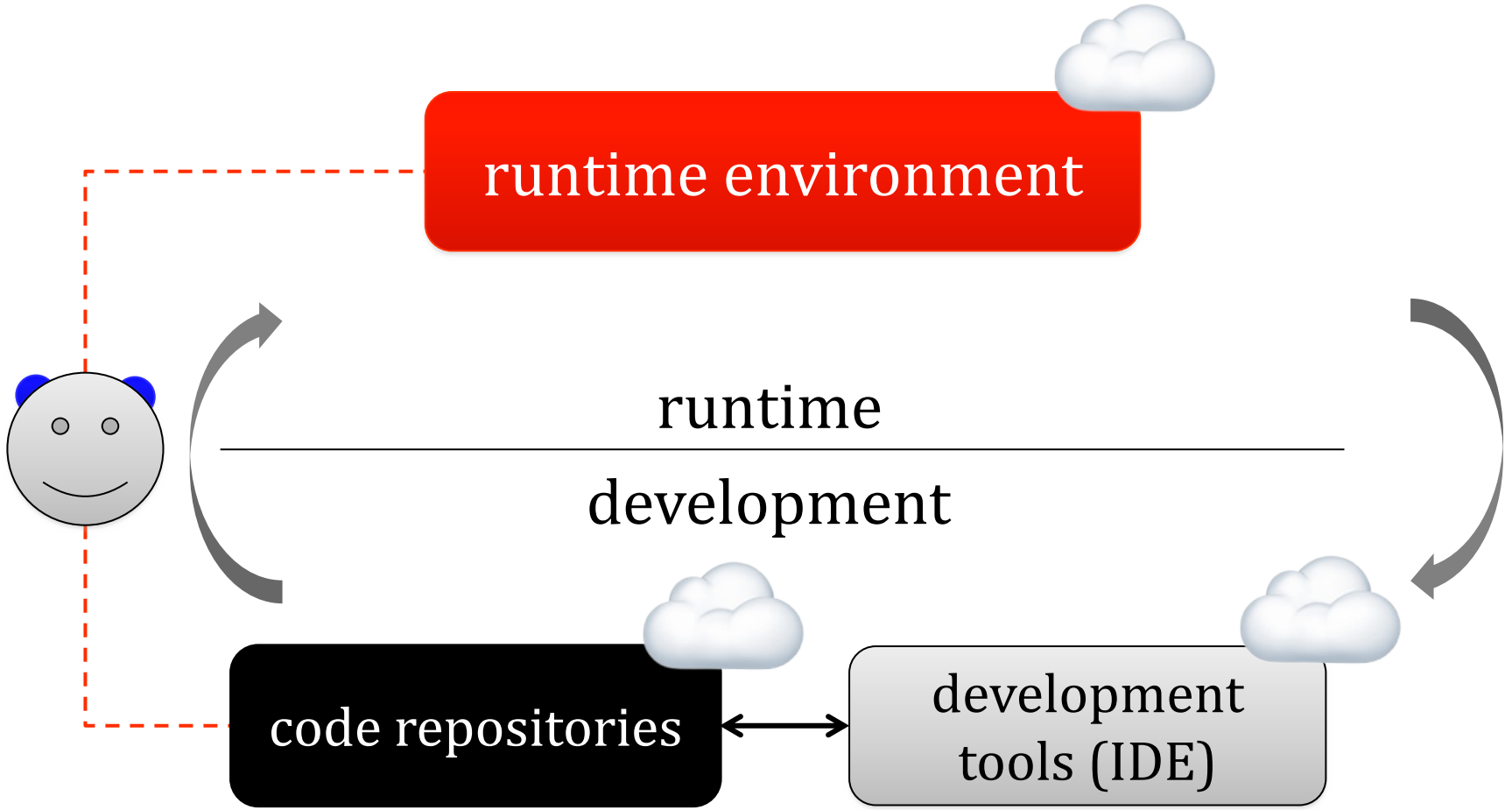
examples – native applications (for desktop):
Anaconda, Aptana Studio, Eclipse, Emacs, IntelliJ IDEA, JetBrains, NetBeans, PHPStorm, PyCharm, Visual Studio (Code), WebStorm, Xcode

cloud computing-based solutions:
CodeSandbox, Koding, OpenShift Dev Spaces, Replit, etc.

S. Buraga, *Code in "clouds"*: **www.slideshare.net/busaco/cu-codul-n-nori**

advanced

traditional approach

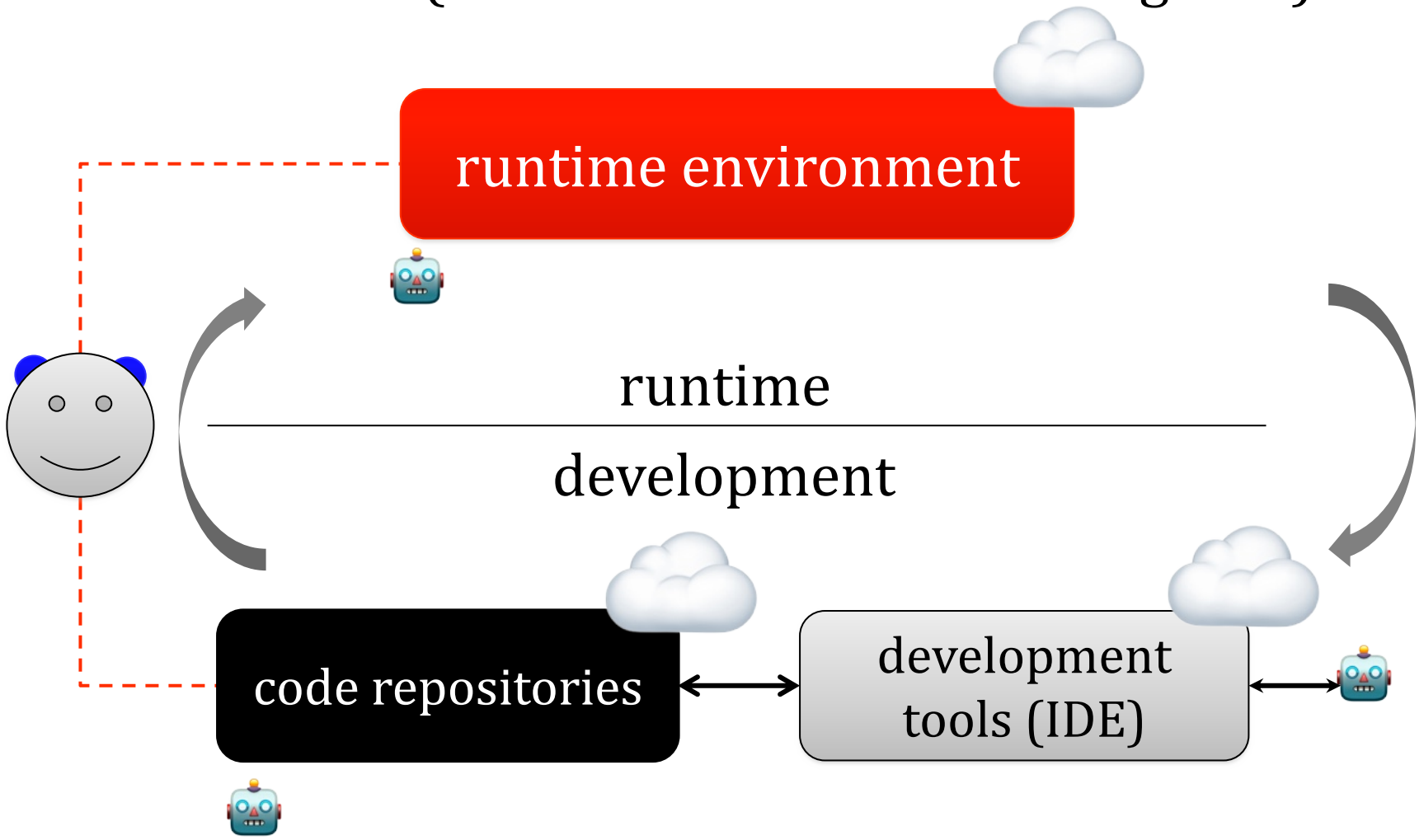


A. Iqbal, M. Haunsenblas, S. Decker (2012)

Development as a Service

advanced

modern approach
– with GenAI (Generative Artificial Intelligence)

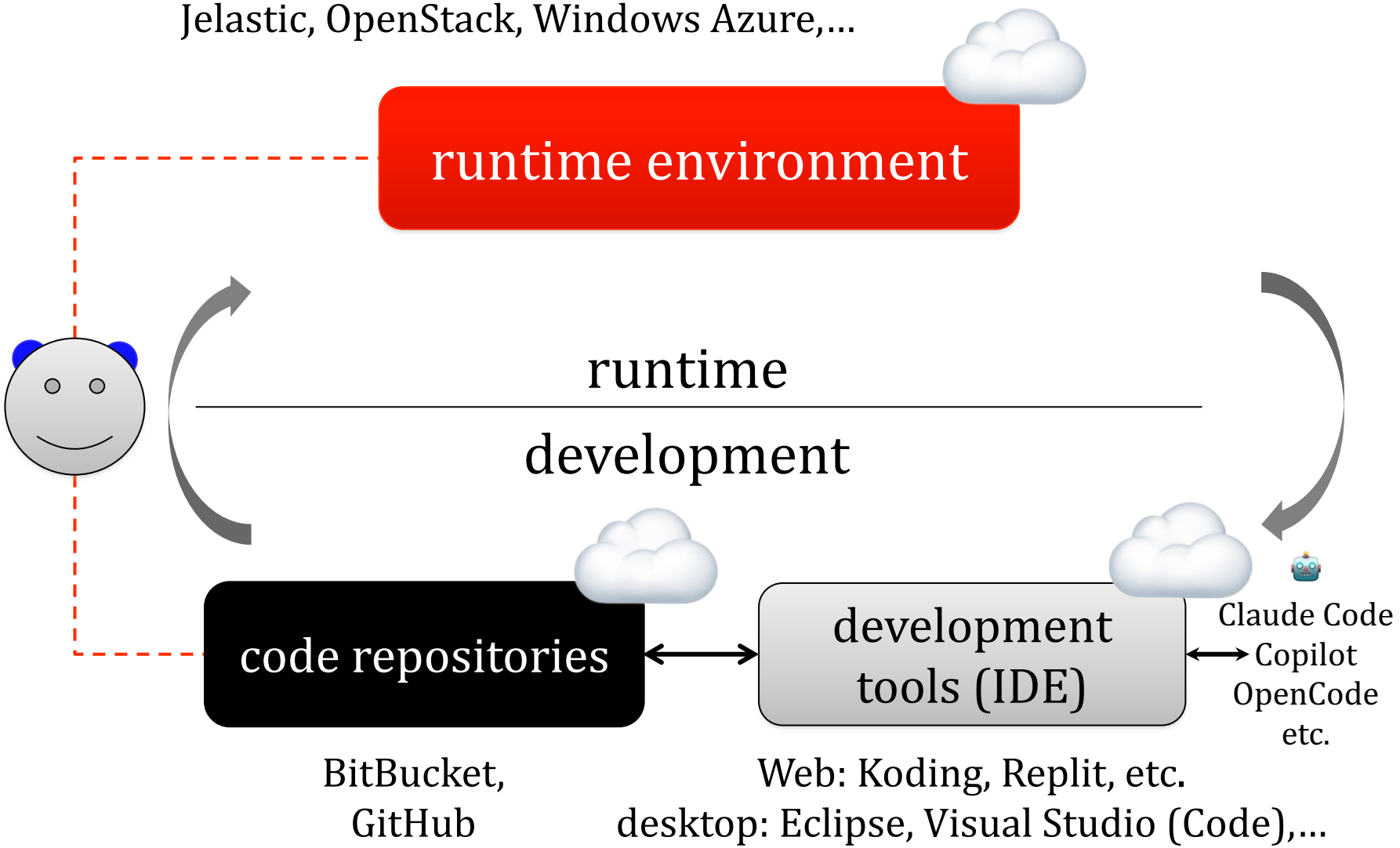


A. Iqbal, M. Haunsenblas, S. Decker (2012)

GenAI tools based on MLLMs = (Multimodal) Large Language Models

advanced

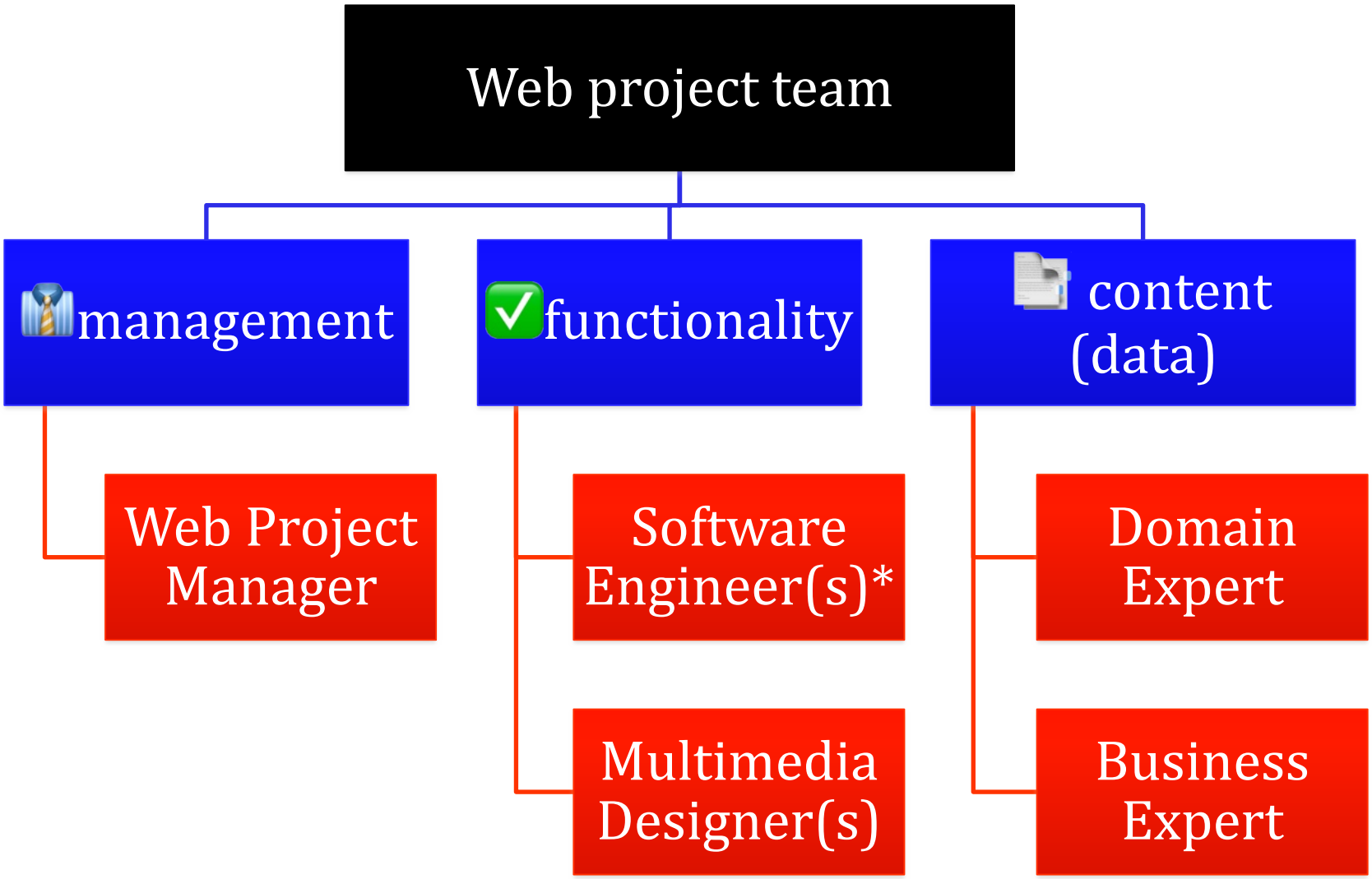
DigitalOcean, Google Cloud Platform, Heroku,
Jelastic, OpenStack, Windows Azure,…



useful tools at **github.com/ripienaar/free-for-dev**

parameters of a web project

main objective
duration
cost
methodology
technologies
processes
outcome
human resources
team profile



*__frontend__ or __backend__ or __full-stack__ (frontend & backend)

www.slideshare.net/busaco/sabin-buraga-dezvoltator-web-n-2019

Several examples regarding Web application architectures?

case study: flickr

Aim:
online sharing of graphical content (photos)

representative social Web application

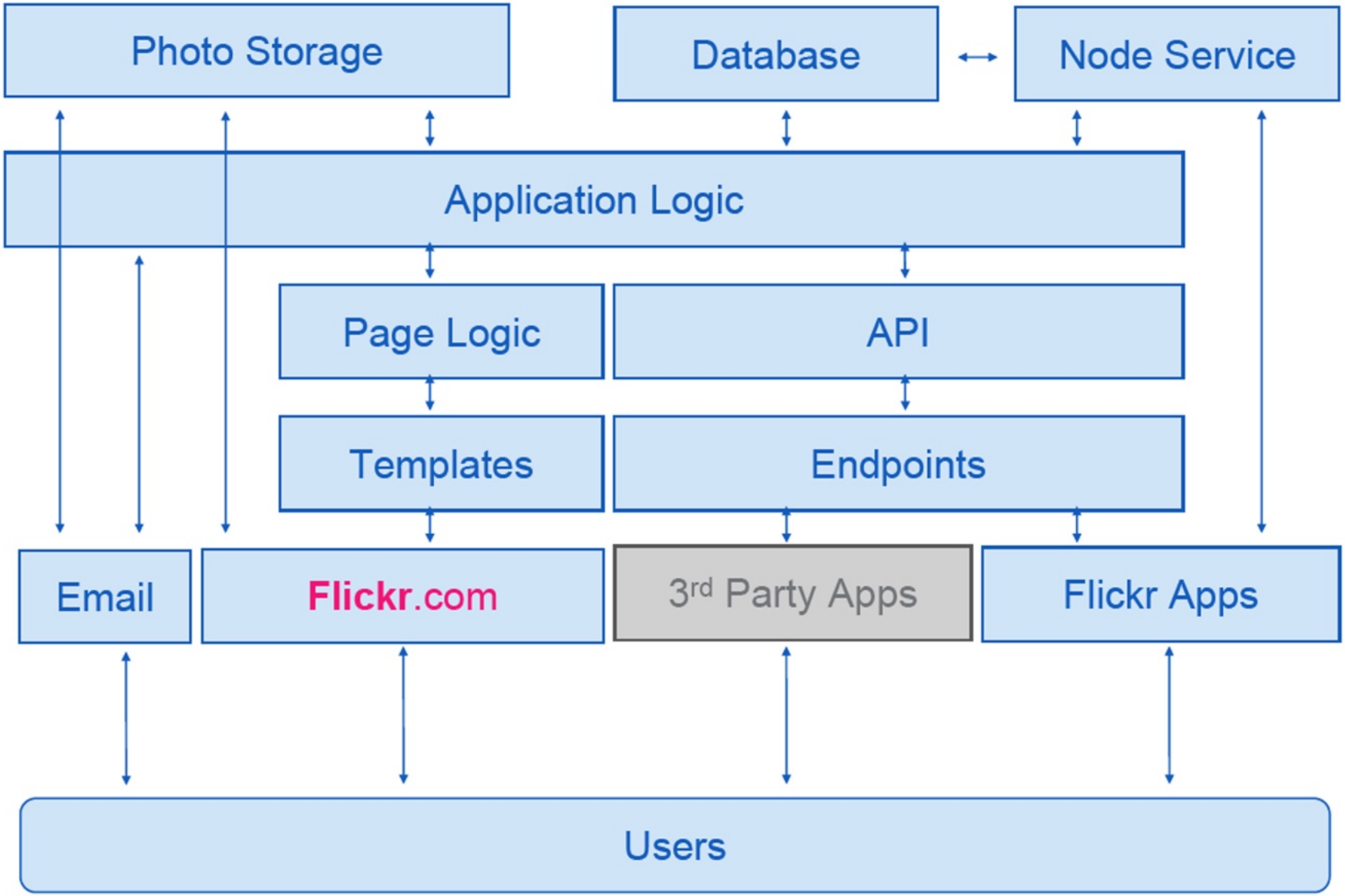
community aggregation – image as social object

support for annotations via tagging
+ comments

case study: flickr – technologies

PHP (processing – application logic, access to API, content presentation via Smarty, e-mail module)
Perl (data validation)
Java (storage node management)
MySQL (data stored in InnoDB format)
ImageMagick (C library for image processing)
Ajax (asynchronous interaction)
Linux (runtime platform)

other details at **highscalability.com/flickr-architecture**



initial architecture – according to Cal Henderson, 2007

generic aspects concerning the system design:

resource categories: *user* + *picture*

relations between *user* instances (e.g., *follow*)

relations between *user* and *picture* instances
(*make*, *depicts*, *comment*, *like*,...)

performance features:
response time, scalable software architecture,
scalable persistent storage, image optimization

recommending resources (*user*/*picture*) of interest

details in the article *Design a Picture-Sharing System (2024)*
www.geeksforgeeks.org/design-a-picture-sharing-system-system-design/

case study: netflix

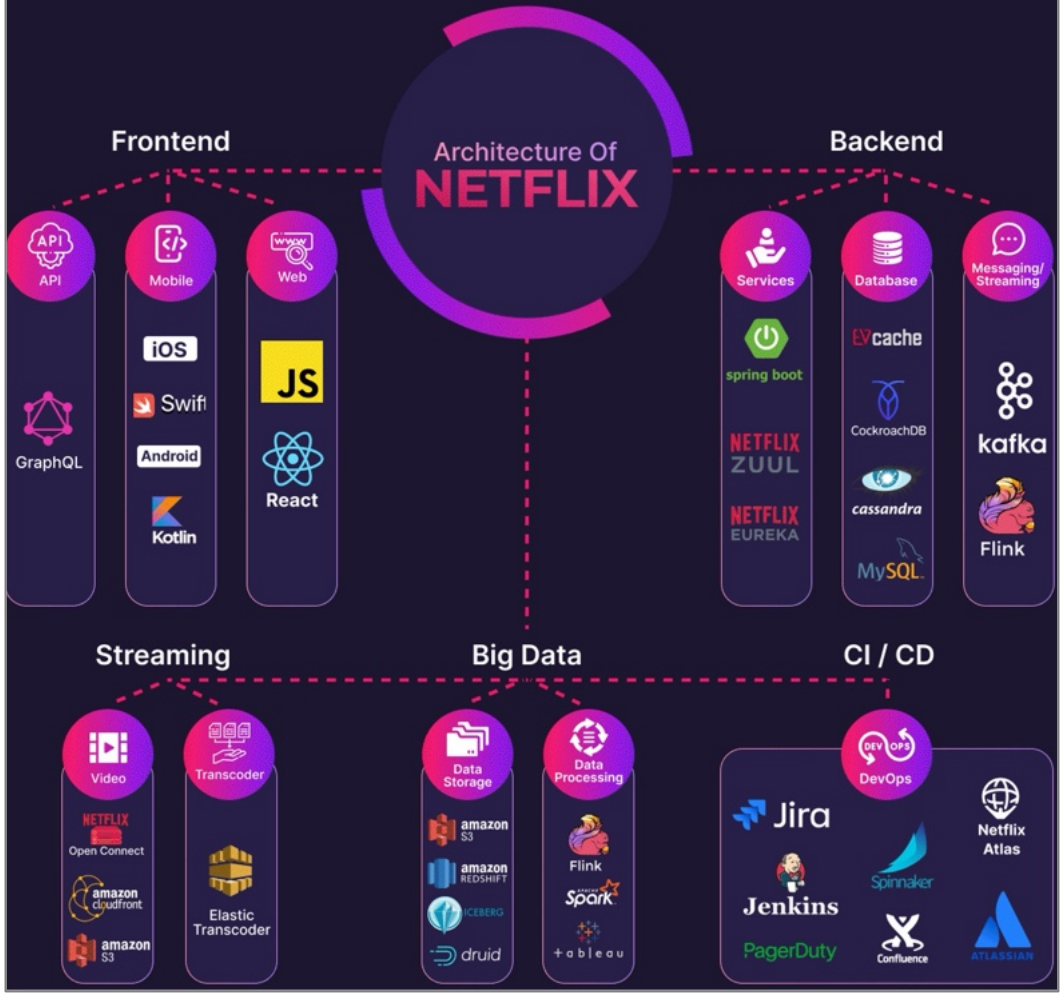
Aim: providing video content on demand (streaming) + Web TV

services available on multiple devices/platforms

cloud-based deployment

adopting, among others, various open technologies

netflixtechblog.com



aspects of interest:
content (movies, TV shows,…),
storage & encoding/decoding (transcoding),
adaptive streaming,
playback (support for heterogeneous devices)

C. Desai (Cloudairy, 2023)

backend processing	Flask (Python), Java, Node.js
frontend processing	React, WinJS (JavaScript)
storage systems	MySQL, PostgreSQL, Oracle, AtlasDB, Cassandra, Hadoop, Presto, etc.
cloud services	Amazon EC2 (video processing) Amazon S3 (storage)
SQL services	Amazon RDS (Relational DB Service)
NoSQL services	Amazon DynamoDB
code management	GitHub (implemented in Ruby + C)
continuous integration	Jenkins (Java implementation)
server management	Apache Mesos (written in C++)
content distribution networks	Open Connect CDN (FreeBSD, Nginx), Akamai, Level 3, Limelight
monitoring	Boundary, LogicMonitor, Vector,…

highscalability.com/a-360-degree-view-of-the-entire-netflix-stack/
stackshare.io/netflix/netflix

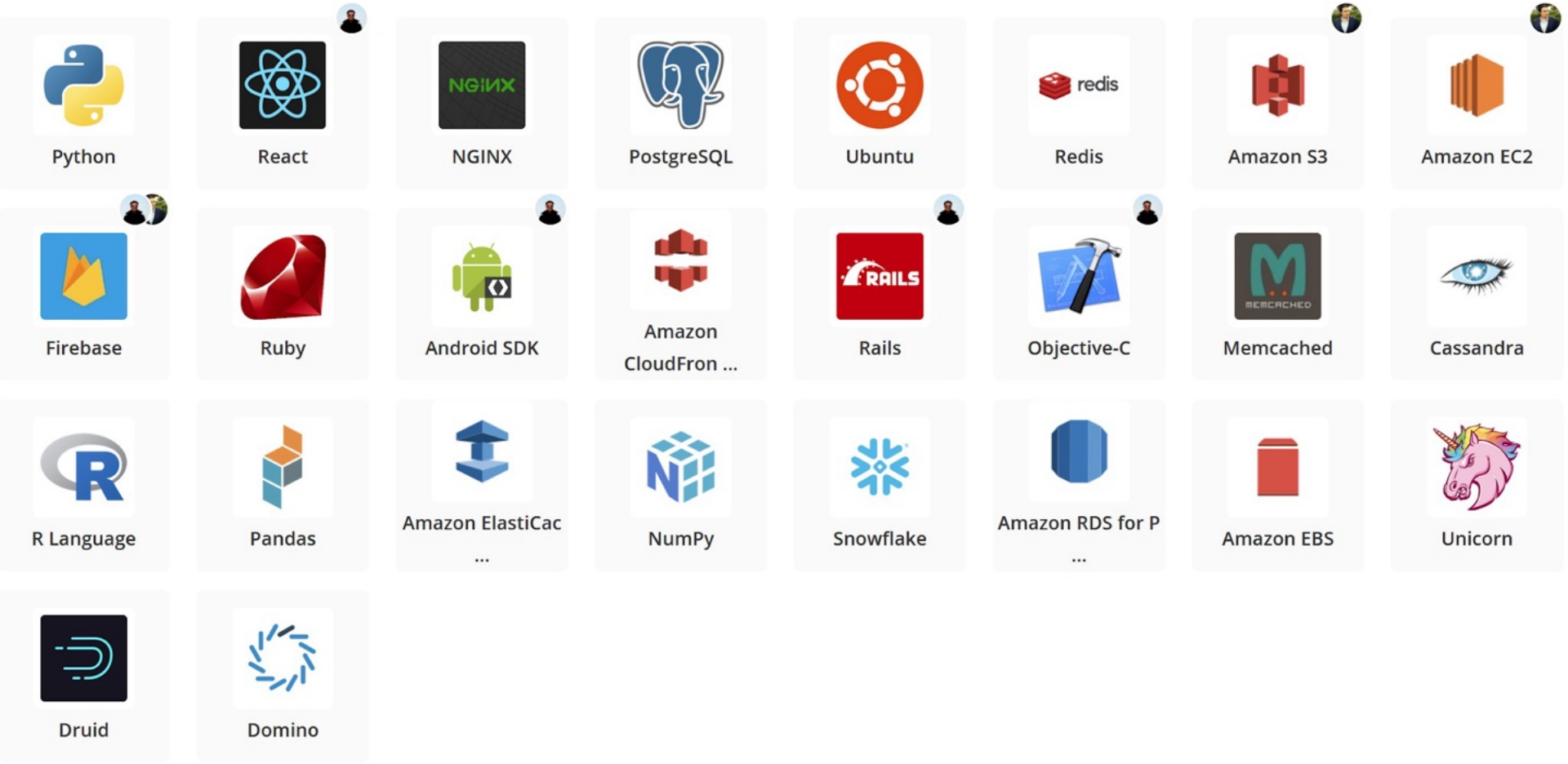
case study: instacart

Purpose: multi-platform Web application offering fast home delivery services for grocery products

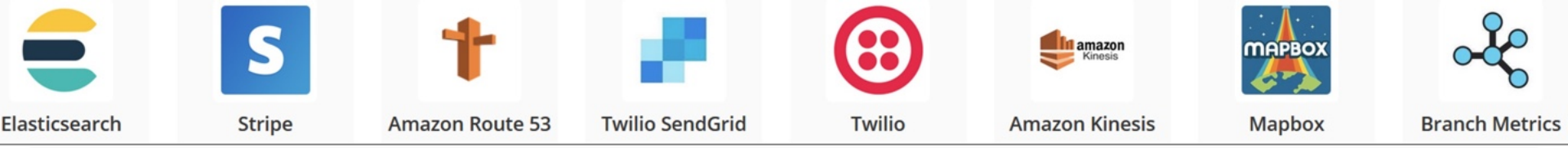
products: 500 millions
local shops (tenderers): 40 thousands
localities: 5500 (US + Canada)
orders: millions/year

stackshare.io/instacart/instacart/

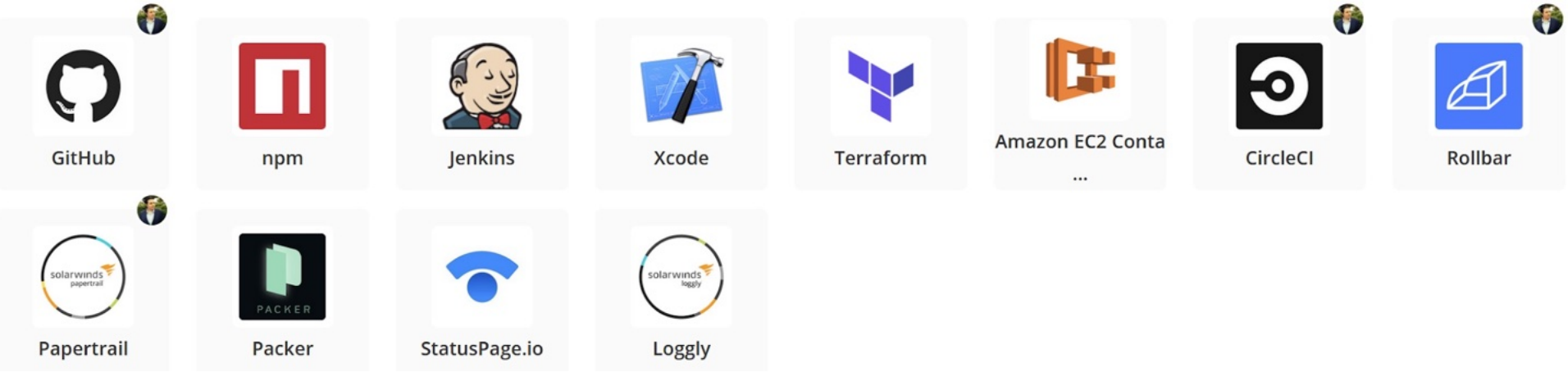
case study: instacart



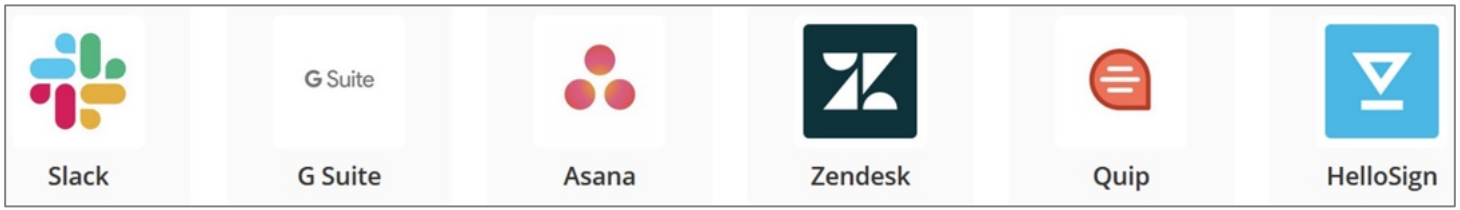
technologies + main tools – application & data



additional instruments, tools – utilities



tools for development operations – DevOps



internal processes regarding the business – business tools

Web application	Backend processing	Persistent storage
Amazon	Perl, Java	MySQL (MariaDB), Amazon DynamoDB, Amazon SimpleDB, Amazon ElastiCache
Coursera	Django (Python), Node.js (JavaScript), Play (Scala)	MySQL, Apache Cassandra
DuckDuck Go	Node.js	PostgreSQL
Facebook	Hack, PHP (HHVM), Tornado (Python), Java, JavaScript	RocksDB, Presto, Cassandra, Beringei
Forbes	ASP.NET, Node.js, PHP 7, Ruby	MySQL (MariaDB)
Google	C++, Dart, Go, Java, Python	BigTable, MariaDB

stackshare.io/stacks

Web application	Backend processing	Persistent storage
Linkedin	Grails (Java), JavaScript, Scala	MySQL, Oracle DB, RocksDB, Hadoop
Lyft	PHP, Flask (Python), Java, Go, C++	MongoDB, DynamoDB, Redis
Medium	Node.js, Go	Neo4j, DynamoDB, Redis
Pinterest	Django (Python), Java, Go	MySQL, Hadoop, Apache HBase, Redis, Memcached
Stack Overflow	.NET Framework (C#)	MS SQL Server, Redis
Sound Cloud	Clojure, Scala, JRuby	MySQL

stackshare.io/stacks

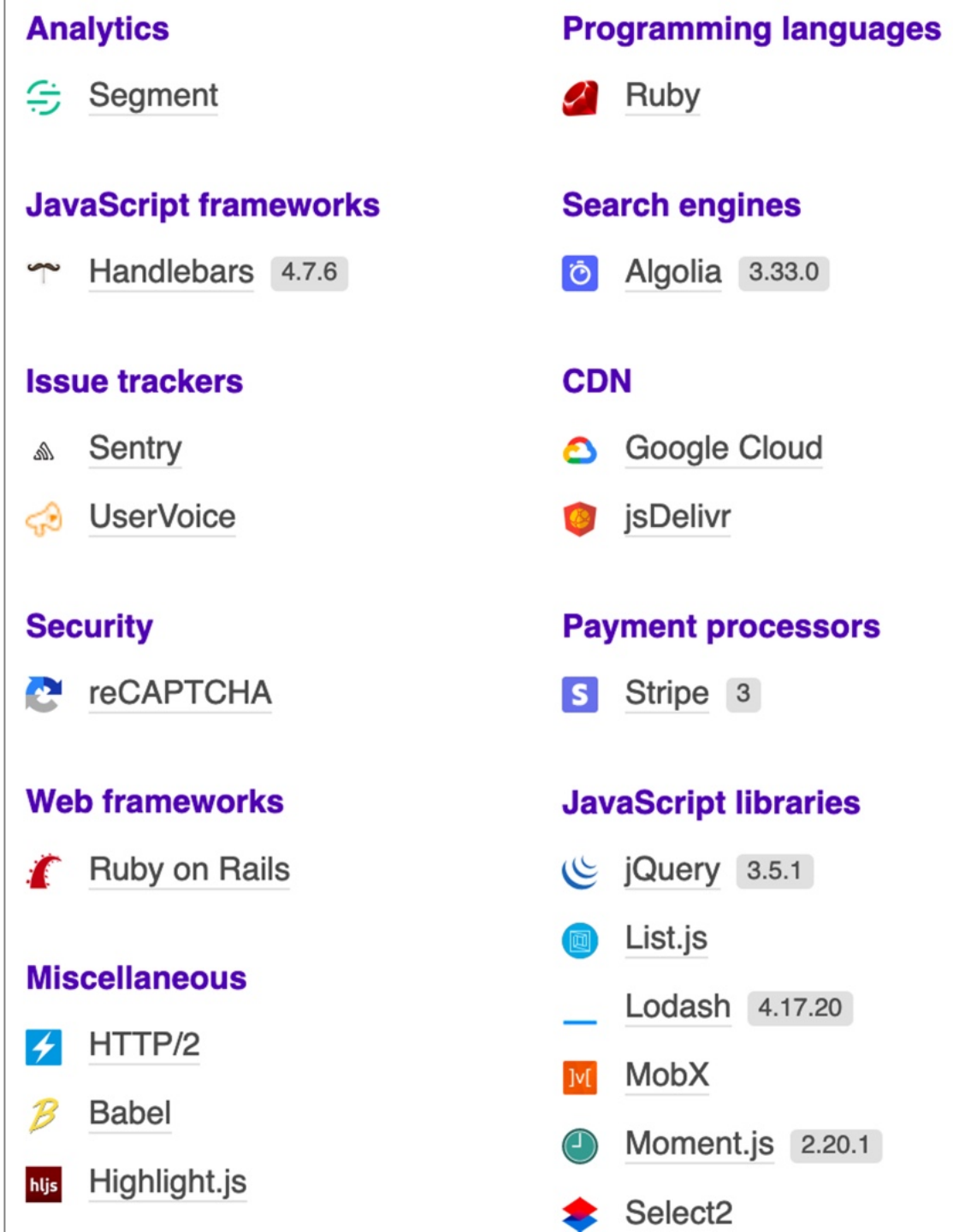
Inspecting the technologies used by a Web application with the instruments

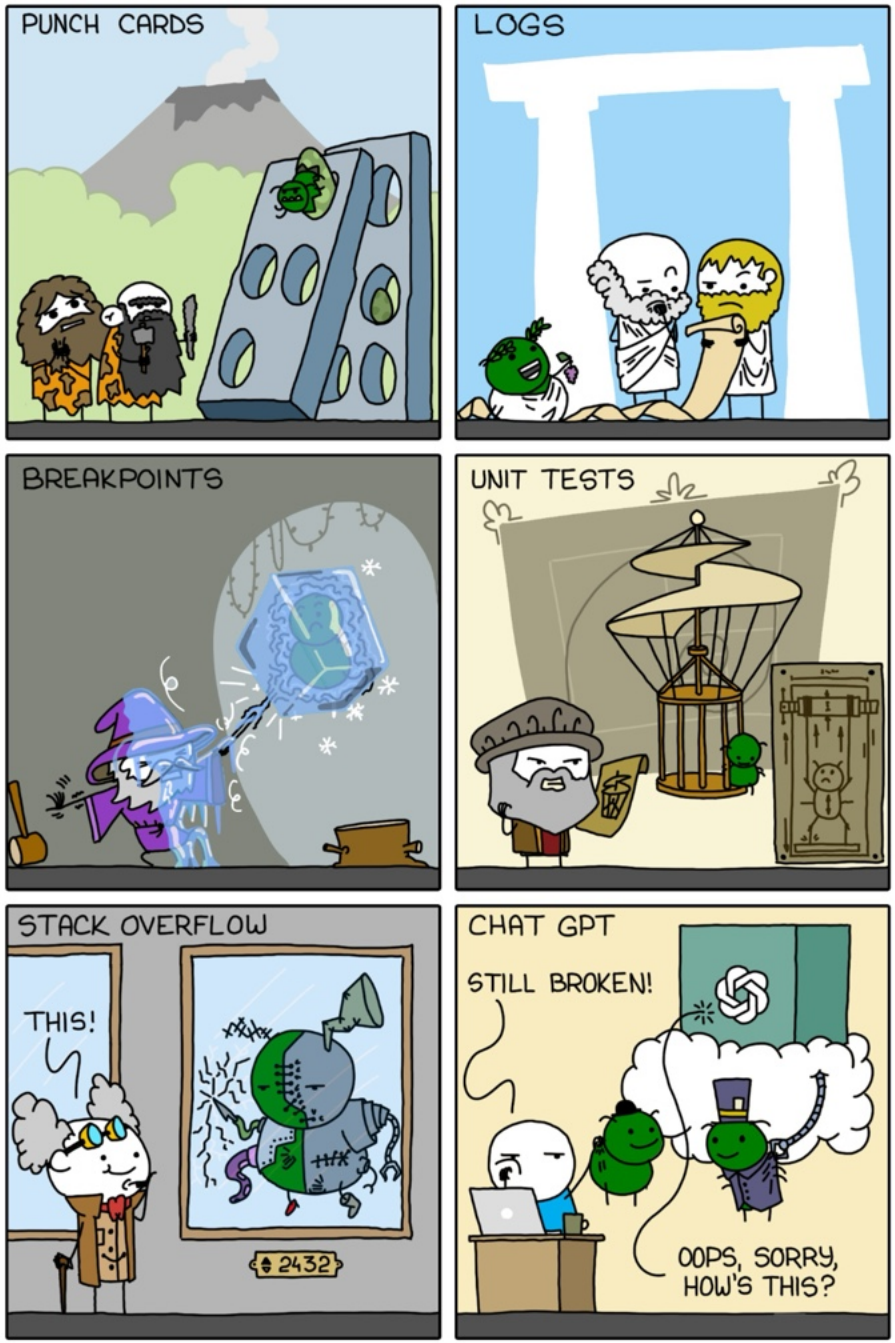
Wappalyzer

www.wappalyzer.com

WhatRuns

www.whatruns.com



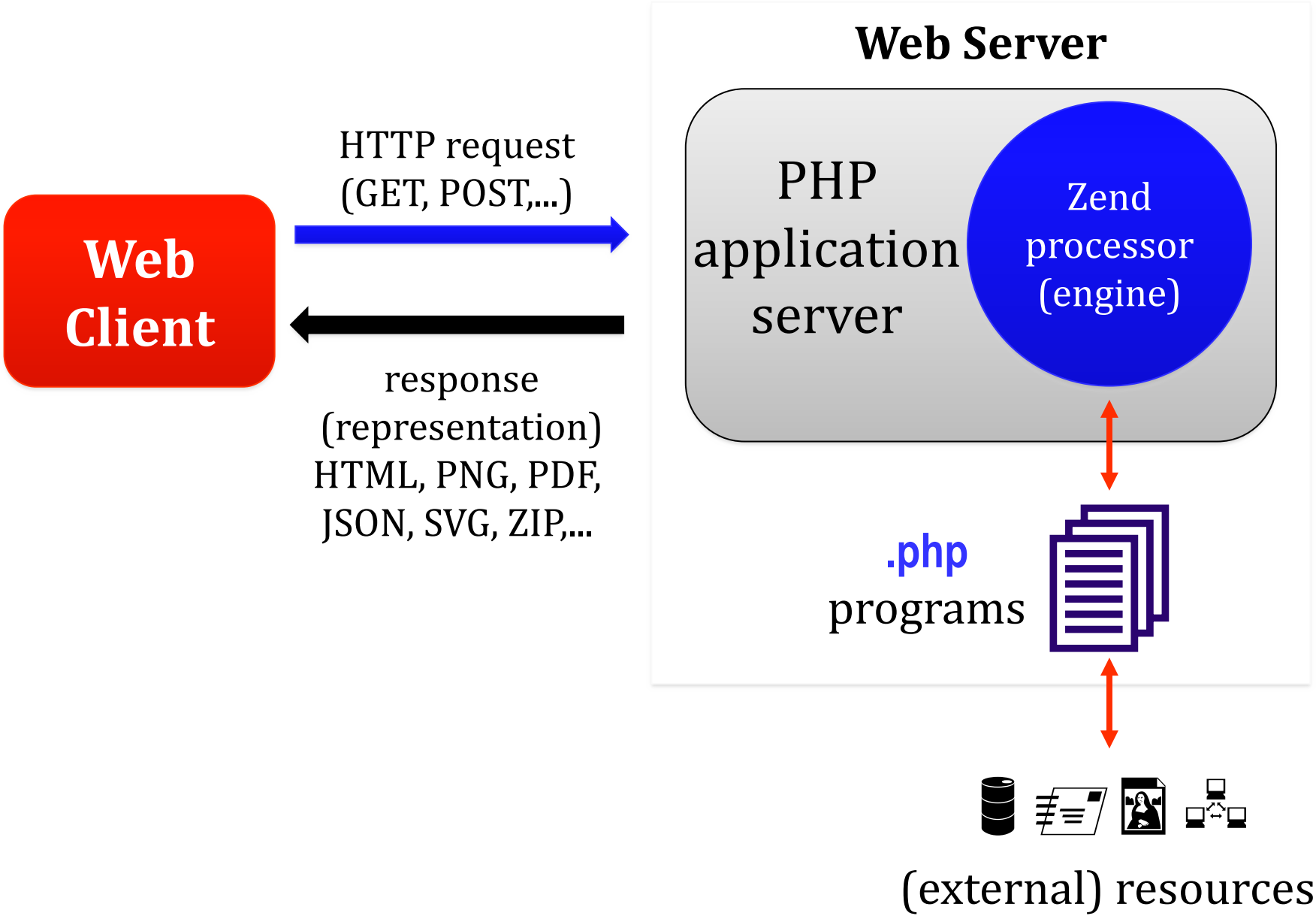


summary

Web programming

- **Web engineering**

Web application development – essential aspects



next episode:

Web application development in PHP